

國立成功大學
電機工程研究所
碩士論文

針對邊緣運算裝置之高能量效率通用計算圖形處理器之
硬體動態使用率功率管理

**An Energy-Efficient General-Purpose Computing on GPU for
Edge Computing Devices Using Hardware Runtime Utilization**

研究生：張哲彬

Student: Che-Pin Chang

指導教授：邱瀝毅 博士

Advisor: Lih-Yih Chiou, Ph.D.

**Department of Electrical Engineering
National Cheng Kung University,
Tainan, Taiwan, R.O.C
Thesis for Master of Science**

July, 2019

中華民國一百零八年七月

國立成功大學

碩士論文

針對邊緣運算裝置之高能量效率通用計算圖形處理器
之硬體動態使用率功率管理

An Energy-Efficient General-Purpose Computing on GPU
for Edge Computing Devices
Using Hardware Runtime Utilization

研究生：張哲彬

本論文業經審查及口試合格特此證明

論文考試委員：

陳中和

郭敬宏

邱滙毅

指導教授：

邱滙毅

系(所)主管：

楊宏澤

中 華 民 國 108 年 7 月 23 日

An Energy-Efficient General-Purpose Computing on GPU
for Edge Computing Devices
Using Hardware Runtime Utilization

By

Che-Pin Chang

A thesis submitted to the graduate division in partial
fulfillment of the requirement for the degree of
Master of Science

at

National Cheng Kung University
Tainan, Taiwan, Republic of China
July 2019

Approved by :

C. H. Chen

Chia-Hung

Leiyi Chion

Advisor :

Leiyi Chion

Chairman :

Ang-Jen Yang

針對邊緣運算裝置之高能量效率通用計算圖形處理器之硬體動態使用率功率管理

研究生：張哲彬 指導教授：邱瀝毅 博士

國立成功大學電機工程研究所 VLSI/CAD 組

摘 要

隨著人工智慧(Artificial Intelligent)、物聯網(Internet of Thing, IoT)與 5G 通訊等技術的發展，數據量越來越龐大，邊緣運算(Edge Computing)的概念也因此被廣泛地討論。邊緣運算主要的概念是將數據在邊緣端預先做部分處理，如此可以降低整體數據處理的時間同時也能減少資料傳輸至雲端所需的時間。在這樣的背景之下，邊緣運算裝置也成為市場關注的重點之一。在運算能力方面，已經被廣泛運用在各類應用程式之中的通用計算圖形處理器，其大量的運算單元及能夠處理大量平行運算的能力讓它成為選擇運算裝置時的考量之一。而若要設計邊緣運算裝置中的通用運算圖形處理器，受限於邊緣運算裝置有限的能量來源和體積之下，高運算能力不再是首要的設計重點，而是如何能在有限的功耗之下盡量提升運算能力成為重要的考量。

本論文提出一個考量硬體動態使用率之動態管理方法及低功耗設計於通用計算圖形處理器的高能量效率解決辦法。在這個方法中會動態的偵測當前的運算核心使用率來進行不同區塊的動態電壓頻率調整，進而達到高能量效率的目標。根據實驗結果，整體而言平均可以降低 22.3%的功率消耗同時付出 3.2%的效能代價，降低約 20%的能量消耗。

關鍵字：邊緣運算裝置、通用計算圖形處理器、動態電壓頻率調整、功率管理

An Energy-Efficient General-Purpose Computing on GPU for Edge Computing Devices Using Hardware Runtime Utilization

Che-Pin Chang

Lih-Yih Chiou, Ph.D.

Department of Electrical Engineering

National Cheng Kung University,

Tainan, Taiwan, R.O.C

SUMMARY

As the rapid development of artificial intelligent (AI) and internet of things (IoT), the amount of data becomes increasingly huge. Under this situation, the concept of edge computing by using edge computing devices has been widely discussed because it can pre-process data and further reduce latency of transferring data to the cloud server. When computing devices are considered, the general-purpose computing on graphics processing units (GPGPUs) has been widely used because of its strong computing capability. However, when designing a GPGPU for edge computing devices, high computing capability is no longer the main consideration due to limited energy source and devices volume. Instead, high energy efficient and power management become one of main issues.

This thesis proposes an energy efficient GPGPU for edge computing devices using hardware runtime utilization-aware power management scheme with low power techniques. The proposed scheme can detect the utilization of SIMT cores and then dynamically adjust the voltage and frequency of different domains to reduce unnecessary power consumption during runtime. According to the experimental results, the scheme can reduce power consumption by 22.3% with only 3.2% performance overhead on average.

Key words: Edge Computing Devices, GPGPU, Dynamic Voltage and Frequency Scaling (DVFS), Power Management

INTRODUCTION

Edge computing is an emerging technology and grows rapidly. In recent years, the concept of edge computing has been widely discussed, its applications and market are also booming. Edge computing is a distributed computing paradigm. It can pre-process data at edge to disassemble data in the network center node into small parts. Therefore, it can speed up the overall data processing speed, mitigate the problems of long latencies and thereby reduce the cost of data transmission. As the applications like artificial intelligent (AI), internet of thing (IoT) and 5G grow rapidly, the amount of data is getting bigger. The growth of edge computing devices is also predictable.

When computing devices are considered, the general-purpose computing on graphics processing units (GPGPUs) have been widely used because of its strong computing capability. GPGPU is a specialized graphics processor that do not use graphics function and additionally provides a highly parallel software programming method and a large number of hardware units to accelerate some data-intensive applications.

When designing a GPGPU in an edge computing device, since the edge computing device is generally powered by a limited energy source such as a battery, in the case where the battery energy technology does not have a breakthrough development, it is necessary for designer to make trade-offs between power consumption and computational capability. Therefore, how to reduce power consumption without reducing excessive computing power becomes a problem that must be solved.

PROPOSED DESIGN

This thesis proposes an energy efficient GPGPU power management mechanism for edge computing devices. The design considers the runtime hardware utilization for power management and uses the low-power technique like clock gating to reduce overall power

consumption. The goal of power management is to find out the critical blocks in the current operation. By increasing the voltage/frequency (V/F) of key blocks and reducing the V/F of non-critical blocks, the power consumption of non-critical blocks can be minimized without reducing the critical performance, achieving the goal of high energy efficiency.

The proposed design in this thesis mainly includes two parts: clock gating mechanism and runtime hardware utilization power management scheme. The design of clock gating needs to consider the timing of clock gating. This design is to transmit all the enable signal in combinational circuits to an OR gate as a basis that whether the clock gating cell can be turned on or not. The purpose of the OR gate here is to confirm whether the current combinational circuit performs a valid operation.

Based on the basis of the above clock gating technology design, through the ratio of the clock on or off, we can realize if the current application is in core-intensive or memory-intensive trend during runtime. And we define the ratio of the clock on or off as the hardware SM core utilization (in a sampling interval). Therefore, this thesis uses the runtime hardware core utilization rate for power management. When the runtime core utilization rate is high, the current operating state is determined to be core-intensive, and if the core utilization rate is low, it is determined to be memory-intensive. Then, according to the determined trend, the V/F are adjusted for the two clock domains, SM core and DRAM clock domains.

In addition, in order to avoid excessive V/F adjustment and to make the algorithm more scalable in different applications or different architectures, this thesis proposes a finite state machine (FSM) to further optimize power management. This FSM takes advantage of the transition between states to avoid V/F adjustments due to abnormal utilization rate and the ability to adjust the utilization threshold so that the threshold can have different levels with different operating states. And that can make the power management algorithm more adaptable to different situations.

RESULTS AND DISCUSSION

All simulations were performed using CASLAB-GPUSIM complied to HSAIL ISA with the proposed extended features. In the whole GPGPU, PLL (Phase-Locked Loops) [6] is needed to adjust the frequency of the SM Core and DRAM, a DC-DC converter [7] to adjust the voltage value and a PMU (Power Management Unit) to manage the selection of the overall V/F.

According to the experiment results, the proposed design can reduce 22.3% power consumption with 3.2% performance overhead, while the experiment comparison, DTA, reduce 44.7% power consumption with 42.1% performance overhead. Overall, the proposed design can save energy by around 20% and DTA save energy by 4.4% on average.

CONCLUSION

In this thesis, we propose an energy efficient GPGPU for edge computing devices using hardware runtime utilization-aware power management scheme with low power techniques. The proposed scheme can detect the utilization rate of SIMT cores and then dynamically adjust the voltage and frequency of different domains to reduce unnecessary power consumption during runtime. According to the experimental results, the scheme can reduce power consumption by 22.3% with only 3.2% performance overhead on average, and which means the energy consumption is reduced by about 20%.

致 謝

這篇論文能夠順利完成要謝謝我的指導教授 邱瀝毅老師，不管是在課堂上的教學或是研究上的指導，都讓我在專業、知識以及研究方法上都受益良多。在碩士班修業期間，當研究遇上瓶頸或是文書報告的表達不夠完善時，他都能夠有耐心地傾聽，並適時的給予我改進的方向及建議，讓我可以在最後完成自己的研究以及本論文。同時也要感謝口試委員 陳中和教授以及 郭致宏教授，給予我研究論文上的建議及指教，讓這篇論文可以更趨完整。

另外要感謝實驗室的學長姐：良盈、啟睿、侑霖、政威、朝凱、士皇、聖凱、境昱、祈叡、易溥、品修、宗翰、維軒、忠翰，在課堂、研究及生活上給了我許多的建議及幫助。以及感謝我的同學：俊興、健棠、晉維、柏丞，在遇上難題時可以互相討論，也互相勉勵，一起度過碩士生涯的這幾年。而實驗室的學弟妹們：憲譚、峻豪、昱宏、育緯、紀寬、唯宏、錫峰、志展、聖凱、宗蕙、偉碩，謝謝你們在各方面的幫助，實驗室有你們的加入也多了一分活力。

最後感謝我敬愛的家人們及宛儒，有你們的支持與陪伴，讓我在碩士班的這幾年能無後顧之憂的完成學業，願你們與我一同分享這份喜悅，謝謝！

張哲彬 謹誌

中華民國一百零八年七月 台南府城

目錄

摘 要	i
致 謝	vi
目 錄	vii
表目錄	ix
圖目錄	x
第 1 章 緒論	1
1.1 研究背景	1
1.1.1 邊緣運算	1
1.1.2 通用計算圖形處理器	2
1.1.3 高功率消耗之影響與功率管理	3
1.2 研究動機	4
1.3 研究貢獻	5
1.4 論文架構	5
第 2 章 相關研究背景	6
2.1 通用計算圖形處理器整體架構	6
2.2 功率消耗之來源和低功耗技術簡介	8
第 3 章 相關文獻探討	11
3.1 低功耗技術在系統設計中的實現	11
3.2 通用計算圖形處理器之功率管理	14
3.2.1 Fine-Grain DVFS with Execution-Unit Turbo	14
3.2.2 Data-Traffic Aware DVFS	16
3.3 相關文獻總結	18
第 4 章 硬體動態使用率電壓及頻率的動態功率管理	20

4.1 問題描述	20
4.2 目標平台之環境與架構	21
4.2.1 平台介紹	21
4.2.2 多重串流處理器內部架構	23
4.2.3 功率模型	25
4.3 低功耗技術以及功率管理機制	27
4.3.1 低功耗技術的設計	27
4.3.2 硬體動態使用率偵測及電壓頻率調整	30
第 5 章 實驗結果與分析	35
5.1 實驗環境設置	35
5.2 實驗結果	37
5.2.1 時脈閘控	37
5.2.2 硬體動態使用率功率管理	38
5.3 額外硬體付出	43
第 6 章 結論和未來工作	44
6.1 結論	44
6.2 未來工作	45
參考文獻	46

表目錄

表 3-1 相關文獻總結	18
表 5-1 CASLAB-GPUSIM 實驗環境.....	35
表 5-2 功率管理相關實驗環境設定.....	36



圖目錄

圖 1-1 邊緣運算市場大小及預測 [1]	1
圖 2-1 通用計算圖形處理器架構示意圖	6
圖 2-2 功率消耗的組成 [3]	8
圖 3-1 Zacate 系統架構圖 [4]	11
圖 3-2 CPU 與 GPU 整合示意圖 [4]	12
圖 3-3 (a)在不同電壓點對於操作頻率的優化 (b)不同閾值電壓元件的使用率 [4].	12
圖 3-4 Zacate 的多電源區塊 [4]	13
圖 3-5 GPU 系統方塊圖 [6]	14
圖 3-6 Modified-EU 架構圖 [6].....	15
圖 3-7 DTA 系統架構圖 [7]	16
圖 3-8 佇列壅塞情況下的頻率調整方式 [7]	17
圖 4-1 CASLAB-GPUSIM 平台架構圖 [13].....	21
圖 4-2 HSAIL 示意圖	22
圖 4-3 多重串流處理器架構.....	23
圖 4-4 非記憶體元件功率資訊取得流程圖	26
圖 4-5 非記憶體元件功率模型誤差	26
圖 4-6 一般數位電路架構圖	28
圖 4-7 加入時脈閘控設計之電路架構圖	28
圖 4-8 使用時脈閘控設計之模組	29
圖 4-9 功率消耗分布圖	29
圖 4-10 使用率定義之說明圖	30
圖 4-11 CASLAB-GPUSIM 時脈區塊.....	31
圖 4-12 硬體動態使用率功率管理演算法流程	32

圖 4-13 FSM 狀態圖及狀態轉移條件.....	33
圖 5-1 功率管理架構示意圖.....	36
圖 5-2 時脈閘控於各模組中節省之功耗.....	37
圖 5-3 時脈閘控於整體 SIMT Core 中節省之功耗.....	37
圖 5-4 不同取樣時間間隔對於使用率趨勢之影響.....	38
圖 5-5 不同趨勢判定時間間隔對於使用率趨勢之影響.....	39
圖 5-6 多支測試程式之平均使用率.....	40
圖 5-7 功耗比較圖.....	40
圖 5-8 效能比較圖.....	41
圖 5-9 整體功耗比較圖.....	41
圖 5-10 整體效能比較圖.....	42
圖 5-11 整體能量比較圖.....	42





第 1 章 緒論

1.1 研究背景

1.1.1 邊緣運算

近年來邊緣運算(Edge Computing)的概念被廣泛地討論，其相關的應用及市場也正蓬勃發展，如圖 1-1 所示[1]。邊緣運算是一種分散式運算的架構，相對於雲端運算而言，邊緣運算將應用程式、數據處理等運算工作由雲端的網路中心節點移至邊緣節點，在邊緣端可以對數據預先做部分的處理，如此可以將在網路中心節點的大型服務或是數據拆解成小的部分，一方面可以加快整體的資料處理速度，另一方面可以降低資料傳送所需的延遲時間，進而降低資料傳輸的成本。在人工智慧(Artificial Intelligent, AI)、物聯網(Internet of Thing, IoT)與 5G 通訊越來越受市場重視的當下，數據量越來越龐大，邊緣運算裝置的快速成長也是可預期的。

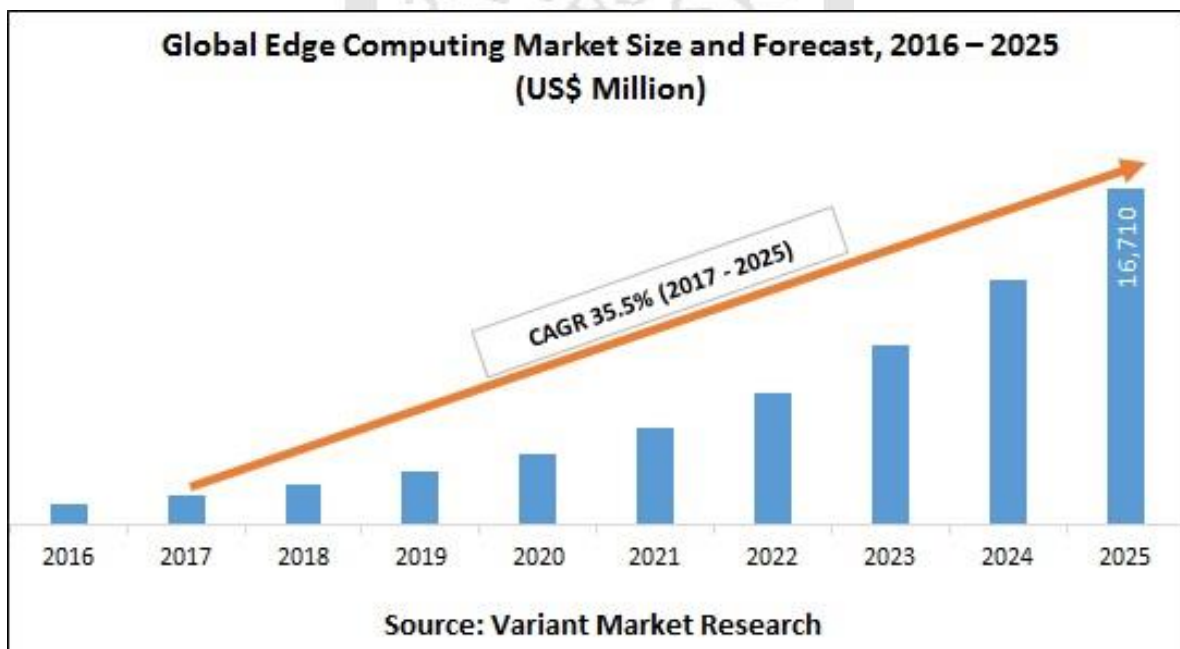


圖 1-1 邊緣運算市場大小及預測 [1]

過去 AI、大數據這類的議題仰賴的是強大的雲端運算能力來進行數據的分析以及演算法的運作。但隨著雲端運算逐漸往邊緣移動之後，不同於雲端的高速運算，邊緣運算裝置受限於裝置體積和有限的能量來源，將不再以追求強大的運算能力為目標。因此，在設計邊緣運算裝置時，需要考慮的是如何在運算能力和功率消耗之間取得平衡，也就是以高能量效率為主要的設計考量。

1.1.2 通用計算圖形處理器

大約在 2010 年代初期，深度學習(Deep Learning)或者說類神經網路(Neural Network)的技術突破帶動了一波人工智慧的浪潮，這其中最大的功臣莫過於通用計算圖形處理器(General-Purpose computing on Graphics Processing Units, GPGPU)晶片的發展了。通用計算圖形處理器是利用處理圖形任務的圖形處理器來計算原先由中央處理器(Central Processing Units, CPU)負責的通用計算任務，通用計算圖形處理器相比於中央處理器最大的優勢在於它在處理單指令多資料流 (Single Instruction Multiple Data, SIMD) 任務時可以有更強大的平行運算能力，所以在類神經網路中多數都是由卷積運算(Convolution)組成的情況之下，通用計算圖形處理器就能得到比中央處理器更多的好處。通用計算圖形處理器這種爆炸性的運算能力成長也讓許多應用紛紛採用通用計算圖形處理器來當運算用的硬體核心，除了上述所提到的深度學習、類神經網路之外，還有像是在 2010 年代中期竄紅的數位加密貨幣比特幣(Bitcoin)、以太坊(ETH)等的“挖礦”行為、自駕車、天氣預測、數值分析等等，在採用通用計算圖形處理器之後都得到了不錯的進展。

通用計算圖形處理器晶片能夠有如此高速的運算能力成長，除了製程的支持之外，也靠著運算核心數量的堆疊及記憶體頻寬的擴展來達成，不過伴隨而來的是大量的功率消耗及熱累積，如何管理龐大的功率消耗並且降低高功耗對晶片的負面影響對晶片設計者來說是非常嚴峻的考驗。

1.1.3 高功率消耗之影響與功率管理

熱效應

對於所有的晶片來說，高功率的另一個負面效應就是廢熱的大量產生。熱會導致整體晶片溫度的提升，而溫度的提升對於晶片的各方面如穩定度、效能以及漏電流都會有所影響。穩定度方面，在操作溫度提高的同時，穩定度中的故障因子也會提升，連帶會導致半導體的使用壽命減低。而在效能方面，高溫同時會提高晶片中信號傳遞延遲的時間，進而導致晶片操作頻率、效能下降或甚至是功能錯誤的狀況發生。最後在漏電流方面，溫度升高時，漏電流所造成之靜態功率消耗會大幅上升，所以熱效應對於整體功率的消耗也會造成極大的影響。

功率管理

隨著邊緣運算的興起，邊緣運算裝置的設計也逐漸成為市場關注的重點。而正如 1.1.1 章節中所提到的，在設計邊緣運算裝置中的通用計算圖形處理器時，由於邊緣運算裝置一般都靠電池等有限的能量來源供電的，在電池能源技術並沒有出現突破性發展的情況下，勢必是要在運算能力和功率消耗之中做出取捨，沒有辦法如 1.1.2 節中一般的通用計算圖形處理器靠著運算核心的堆疊來提高運算能力。因此，如何降低功率消耗卻不降低過多的運算能力成為了必須要被解決的問題。而以往的管理方法通常只針對個別的操作電壓點來做操作頻率的優化，並沒有辦法在運行過程針對運行狀況來做頻率電壓的調整。所以近期開始有研究針對關鍵元件的頻率電壓做優化，最主要的目的在於要將功率消耗集中到關鍵元件之中，並且降低非關鍵路徑(non-critical path)上元件的功率消耗，最終降低整體通用計算圖形處理器的功率消耗。

1.2 研究動機

如前述，邊緣運算的概念成為近年熱門的議題之一，邊緣運算裝置的成長也是可預期的。在設計邊緣運算裝置裡的通用計算圖形處理器時，系統設計的考量不再只是追求高運算能力，而是需要考慮運算能力及功率消耗之間的平衡，也就是以低功耗、高能量效率為目標。

動態功率管理機制是一個有效降低整體系統功率的方法，可以在系統運行過程中調整系統的運作狀況來達到降低功率消耗的目的。這其中通常包含一些低功耗技術，如動態電壓與頻率調整(Dynamic Voltage and Frequency Scaling, DVFS)、時脈閘控(Clock Gating)、電源閘控(Power Gating)等。但通用計算圖形處理器的運算能力來自於多核心以及高平行化的架構，使得通用計算圖形處理器在運行過程中的控制機制和資料流比以往的中央處理器複雜許多，這也造成功率管理上的困難。過往通用計算圖形處理器功率管理機制都是透過一些間接的觀測點來得知現行通用計算圖形處理器的運行狀況，或者透過作業系統及應用程式資訊來做功率管理，這樣的管理方法在應用程式變更或者系統平台設置更換之後就會有效果變差的可能。若能直接從硬體使用狀況來進行功率管理，對於降低通用計算圖形處理器功率是更直接有效的。

綜合上述所說，本論文針對通用計算圖形處理器運行時的硬體動態資訊做偵測，動態對通用計算圖形處理器中不同的區塊依據運行的狀況做開關或是電壓頻率的調整。一般來說，應用程式大致都可以分為運算密集、記憶體存取密集或是趨勢不明顯的狀況，本論文分析硬體動態資訊來判斷目前應用程式的運作趨勢，透過調整頻率及電壓的方式，增加關鍵效能的硬體區塊的效能或是減少非關鍵硬體區塊多餘的功率付出，最終達到減少整體功率消耗的目的。

1.3 研究貢獻

本論文提出一套高能量效率且適用於邊緣運算裝置中通用計算圖形處理器的動態功率管理方法，這套動態功率管理方法具有以下特點：

- ♦ 考量硬體動態使用率，直接有效地判斷硬體的運作情況，進而對不同的硬體區塊做功率管理
- ♦ 考量少數特殊硬體使用情況，避免做出過度地頻率電壓調整
- ♦ 動態使用率閾值調整，具有可調性，可以適應不同的應用情境

1.4 論文架構

本論文在第 1 章中介紹本篇論文的研究背景、研究動機以及研究貢獻。第 2 章講述本篇論文需要的背景知識，包含了通用計算圖形處理器內部的架構、低功率技術介紹等等。第 3 章討論過去一些圖形處理器功率管理的研究，對這些研究做出說明和整理。第 4 章，詳細說明本篇論文所提出的硬體動態使用率功率管理方法。第 5 章則是實驗環境的介紹以及實驗結果的說明和分析。第 6 章針對本篇論文做出結論以及探討未來可以繼續進行的目標和方向。

第 2 章 相關研究背景

本章節將會講述本論文內容所使用到的一些背景知識。在 2.1 節中介紹通用計算圖形處理器的架構，在 2.2 節中則是介紹功率消耗的種類以及低功耗技術的簡介。

2.1 通用計算圖形處理器整體架構

傳統圖形處理器(GPU)是專門用來執行繪圖運算工作的處理器，其最主要的特點是擁有大量的運算核心，對於電腦繪圖演算法可以並列執行大量計算，進而加速繪圖所需之光源計算、動態光影、著色、像素處理等運作。而通用計算圖形處理器則是特化的圖形處理器，也就是不使用圖形處理器的繪圖功能，而另外提供可高度平行化的軟體編程方法以及大量硬體運算單元來加速一些資料密集的非圖形應用程式。

通用計算圖形處理器的運算架構為單指令多資料流架構(Single Instruction Multiple Data, SIMD)，其硬體運算核心單元稱為串流處理器(Streaming Processor, SP)，也就是在圖 2-1 中一條一條的 SIMD lane，多個串流處理器的集合被稱為多重串流處理器(Streaming Multiprocessor, SM)，結合了多個執行緒(thread)而形成了單一指令多線程(Single Instruction Multi Thread, SIMT)運算模式。

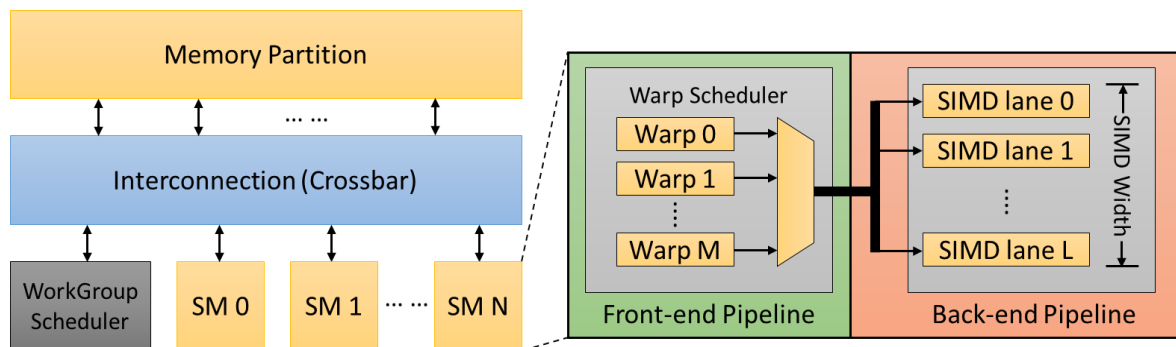


圖 2-1 通用計算圖形處理器架構示意圖

從圖 2-1 中也可以看出整個通用計算圖形處理器的架構。通用計算圖形處理器會以 Grid 為單位，由 WorkGroup Scheduler 接收被指派的任務，在以 WorkGroup(也被稱為 Block)為單位派發任務給 SM 執行。在傳入 SM 之後，會根據 SIMD Width 的寬度(也就是 SP 的數量)將 Thread 包裝成 Warp，在透過 Warp Scheduler 將 Warp 傳入各 SIMD lane 中做處理。其中可以將通用計算圖形處理器大致切為前端管線(Front-end Pipeline)以及後端管線(Back-end Pipeline)。Warp Scheduler 位於整顆 SM 中的前端管線(Front-end Pipeline)的位子，透過管理 Warp 的派發可以讓整個通用計算圖形處理器得到 Thread-level Parallelism 的好處。而各條 SIMD lane 則是在後端管線的地方，在接受到各個 Thread 之後，根據指令類別分別到 Execution Unit 做運算或是到 Load Store Unit 等待記憶體存取，也就是說在後端管線之中可以得到 Data-level Parallelism 或是 Memory-level Parallelism 的好處。以上提到的各種平行度讓通用計算圖形處理器在可平行化的應用程式之中得到大量的效能提升。

在 SM 需要對記憶體部分進行存取時，會透過一個中介網路(Interconnection Network)的機制來進行記憶體存取，所以 Interconnection 就是 SM 以及記憶體之間的溝通橋樑。而在記憶體(Memory Partition)的部分，是由所有的 SM 共用記憶體區塊，這邊的記憶體區塊包含了第二級快取記憶體(L2 Cache)以及動態隨機存取記憶體(DRAM)，所以就是所有的 SM 都會透過 Interconnection Network 來跟 L2 Cache 以及 DRAM 來做資料交換。

2.2 功率消耗之來源和低功耗技術簡介

功率消耗の種類與來源

在現今 CMOS 電路設計之中，電路中的總功率消耗可以分成兩個主要的類別：靜態功耗(Static Power)以及動態功耗(Dynamic Power)，公式如下：

$$P_{Total} = P_{Static} + P_{Dynamic} \quad (2.1)$$

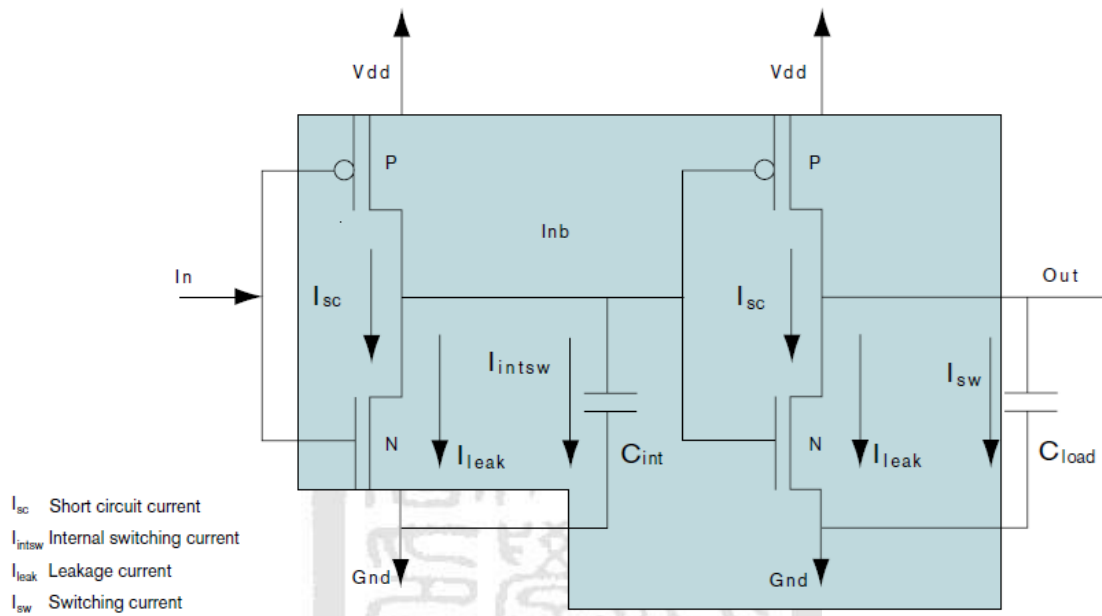


圖 2-2 功率消耗的組成 [3]

靜態功耗主要是由漏電流引起的功耗，而漏電流其中一個產生的原因是二極體反向偏壓的漏電流(reverse-biased diode leakage)，而最主要的原因則是源極至汲極的次臨界漏電流(source-to-drain sub-threshold leakage)，如圖 2-2 所示。靜態功耗公式如下：

$$P_{Static} = V \times I_{leak} \quad (2.2)$$

動態功耗主要是由開關功耗(Switching Power)及內部功耗(Internal Power)組成的，開關功耗是電路在開關過程中對於輸出端的負載電容充放電所產生的功耗，其公式表示如下：

$$P_{Switch} = \frac{1}{2} \times C_{load} \times V^2 \times f \quad (2.3)$$

而內部功耗則是由兩個部分組成，第一個部分是電路在開關過程中對內部電容充放電所產生的功耗，第二個部分則是短路功耗，由於輸入點電壓在實際情況中，電壓上升或是下降都會有一定的時間，在電路輸入端的電壓上升或是下降的過程中，會有某個範圍的電壓值是 PMOS 和 NMOS 都導通的狀況，這時就會有電源直接接到地的直流導通電流，這就是短路功耗。整體內部功耗公式表示如下：

$$P_{Internal} = \left(\frac{1}{2} \times C_{int} \times V^2 \times f\right) + (V \times I_{sc}) \quad (2.4)$$

低功耗技術簡介

CMOS 電路的低功耗技術種類繁多，在以下內容中會簡介在本篇論文中提到的低功耗技術以及在設計電路中常見的一些低功耗技術。

- (1) 時脈閘控(Clock Gating): 是一種在同步序向邏輯電路(Synchronous Sequential Circuit)中的技術，時脈閘控會在時脈樹(Clock Tree)中增加一些邏輯閘，在不需要的時候將時脈關閉，而達到降低動態功率消耗的結果。
- (2) 動態電壓與頻率調整(Dynamic Voltage and Frequency Scaling): 在系統運行過程之中，不同區塊對於當前的應用程式有不同的速度要求，因此在蒐集完系統當前資訊之後，對於關鍵元件給予較高的頻率以及電壓，而對非關鍵元件則是降低頻率和電壓，最終降低整體系統的功耗。
- (3) 電源閘控(Power Gating): 電源閘控透過關停工作區塊的電源而可以進一步省去靜態漏電流的功耗，但設計上也需要付出不少代價，在關停電源復電之後會需要付出一段時間代價，另外在開關之間，電路沒有辦法接收這些忽高忽低的電壓值，會需要隔離器(Isolator)來做應對，面積表現上也會付出代價。
- (4) 多閾值電壓元件設計(Multi- V_{th} Cell): 對於 CMOS 電路來說，低閾值元件(Low- V_{th} Cell)為高速元件，但低閾值元件同時也有高漏電流的缺點，而高閾值元件(High- V_{th} Cell)則是相對低速、低漏電流元件，所以在系統設計時在

非關鍵路徑上使用高閾值元件，將關鍵路徑上才換上低閾值元件，是一種可以滿足既有的時序要求同時降低漏電流的設計方法。

- (5) 其他：其餘低功耗技術包含多電源設計(Multi-Voltage)、Data Gating、Memory Gating、SLEEP Transistor Technique、Emerging Memory 等。

綜合上述所提到的，低功耗技術的核心概念就是關鍵的地方可以高功耗，而非關鍵區域就要盡可能的節省功率的消耗，該用的多用，不該用的就減少支出或甚至停止支出。而設計低功耗電路時的難點在於如何在運行過程中判斷出哪邊是現在的關鍵區域或是哪邊是目前沒有在使用的區塊，以及實作低功耗電路時所需要多付出的代價 (performance overhead, power overhead, area overhead 等) 是否有小於降低功耗所得到的好處。



第 3 章 相關文獻探討

在這個章節之中會探討與低功耗技術和通用計算圖形處理器的功率管理相關的文獻。在 3.1 節之中會先討論低功耗技術在系統設計中的實現，在 3.2 節中則是討論在通用計算圖形處理器中的功率管理方法。

3.1 低功耗技術在系統設計中的實現

文獻(JSSC'12)[4]是一篇實現低功耗技術於移動運算裝置(mobile computing device)系統晶片設計之中的論文。這篇論文主要是在探討在 AMD Fusion APU – Zacate 這顆晶片上所使用的各種低功耗技術，這顆晶片是一個系統單晶片(System-on-Chip, SoC)，其中包含了兩顆 BOBCAT x86 處理器、AMD Radeon™ 6310 DirectX® 11 GPU、Universal Video Decoder (UVD)等，整體架構如圖 3-1 所示。

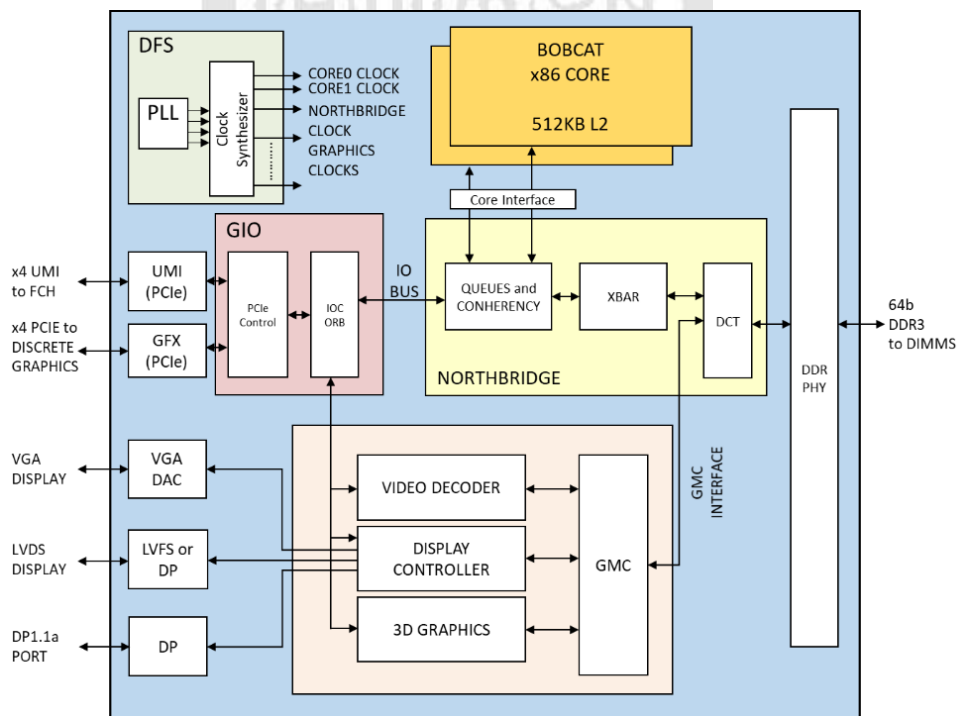


圖 3-1 Zacate 系統架構圖 [4]

在這顆 SoC 中使用了許多方法來降低功耗，首先，在過去 AMD 的系統晶片設計之中，CPU、GPU 兩顆晶片之間的溝通是透過高速 PHY 這個物理接口來完成，但這樣在影片播放等 GPU 運算需求量大的時期，光是 PHY 這個部分就消耗超過 1W 的功

率，因此作者就將 CPU 與 GPU 整合成同一顆晶片，如圖 3-2 所示，避免 PHY 的耗能也同時可以節省面積並且讓 CPU 與 GPU 之間的傳輸 bandwidth 可以提升。

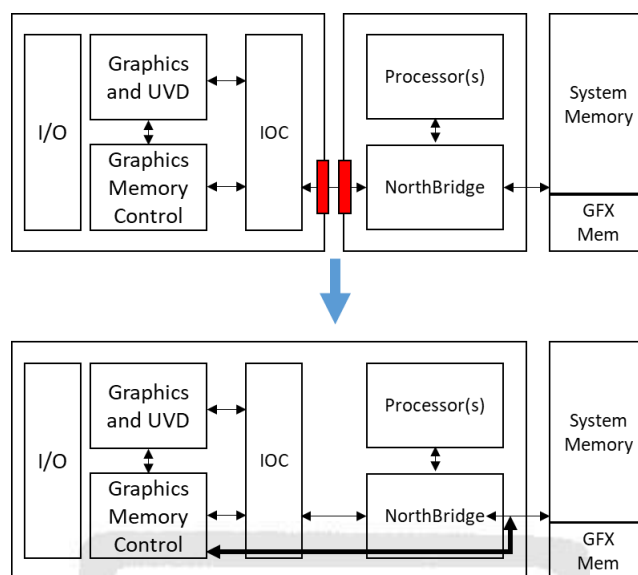


圖 3-2 CPU 與 GPU 整合示意圖 [4]

作者也在 CPU 和 GPU 上個別使用了 DVFS 機制來降低功率消耗，這裡的 DVFS 是在不同的操作電壓點，在不影響功能的前提之下降低操作頻率，盡可能達到最好的效能功率比(performance/power)，如圖 3-3 (a)。另外，作者也採用多閾值電壓元件設計的低功耗技術，在關鍵路徑(critical path)上的元件都替換成 LVT(低閾值電壓)元件，非關鍵路徑上則是採用 HVT(高閾值電壓)元件或是 SVT(一般元件)，整體元件使用率如圖 3-3 (b)。

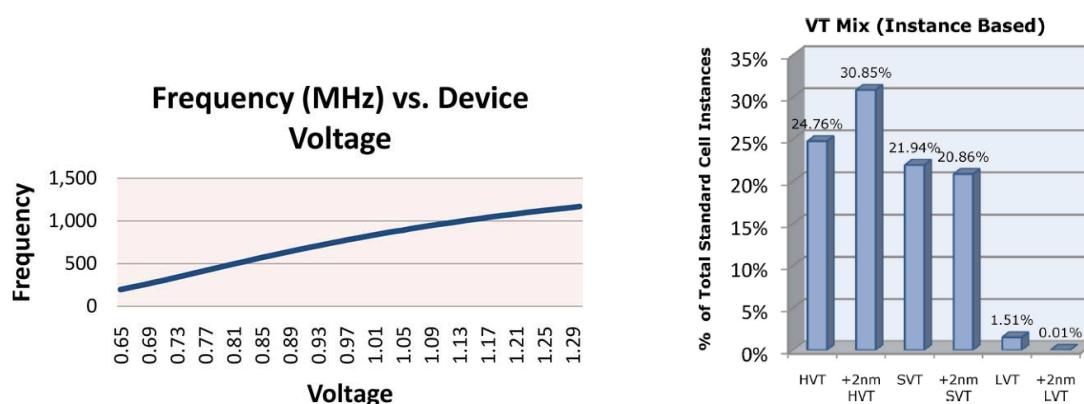


圖 3-3 (a)在不同電壓點對於操作頻率的優化 (b)不同閾值電壓元件的使用率 [4]

最後，作者對於電源的部分也作了許多低功耗的處理，如圖 3-4 的多電源區塊規劃。在整顆 SoC 上切出了多個電源區塊(power domains)同時也使用了電源閘控的技術。根據當前應用程式的運行狀況可以給予不同區塊不同的電壓值，在區塊長時間不會被使用的時候也可以將電源直接關掉，進一步節省漏電流的產生。另外，對於電源閘控也放上 HVT PFET 來隔離電源與電路，避免電源頻繁的開關造成電路的功能異常。

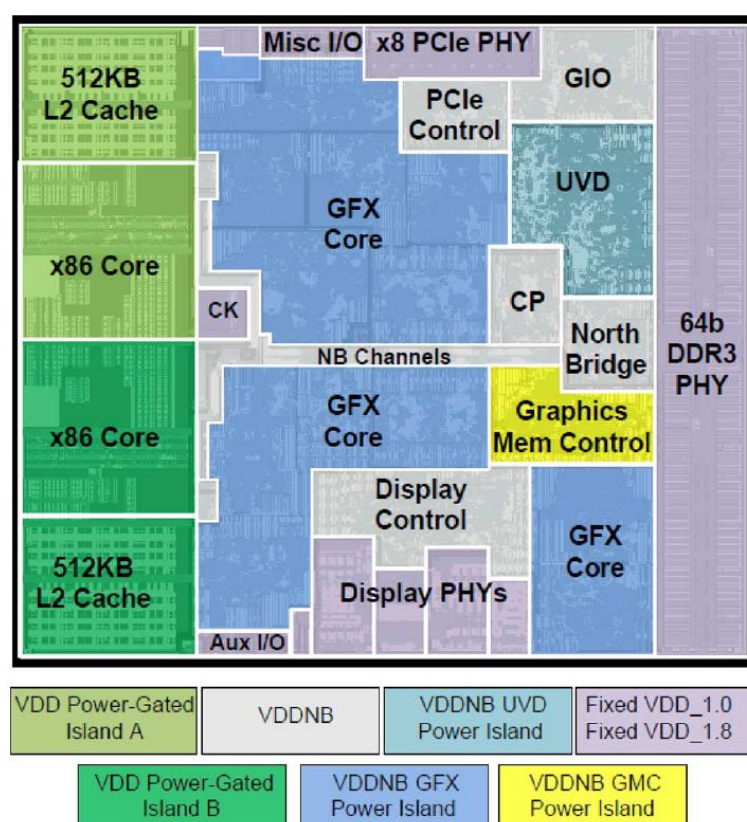


圖 3-4 Zacate 的多電源區塊 [4]

3.2 通用計算圖形處理器之功率管理

3.2.1 Fine-Grain DVFS with Execution-Unit Turbo

在文獻[6]這篇論文之中，作者提出了一個細緻的 DVFS (fine-grain DVFS)方法，搭配上其他的技術如 retentive sleep 和 reliable-aware wakeup power gates 等，而達到高能量效率的目的。

Fine-grain DVFS 機制指的是更細微的 DVFS 操作，可以針對較小的區塊作頻率電壓的調整，在這篇論文之中，作者是將 fine-grain DVFS 機制使用在功率效能比 (power/performance)最關鍵的區塊，也就是使用在最主要的運算單元 Execution Unit (EU)之上。整體的系統方塊圖如圖 3-5。

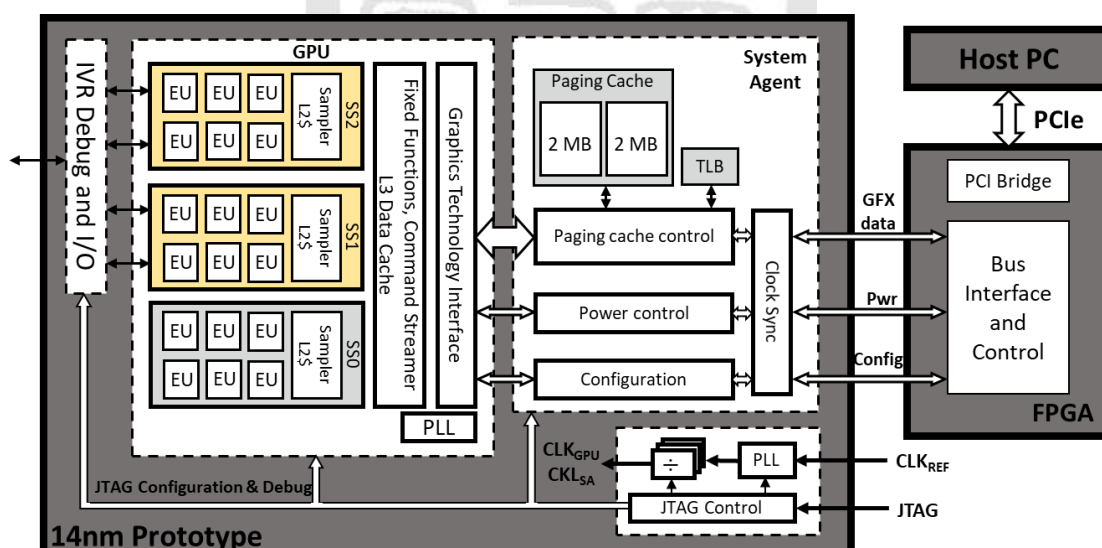


圖 3-5 GPU 系統方塊圖 [6]

GPU 在運行期間的工作負荷(workload)時常在變動，而這篇論文主要針對的情境是 EU-dominant 的 workload。在 EU-intensive 時期，fine-grain DVFS 機制會將 EU 區塊的操作電壓利用整合的電壓調節器(Integrated Voltage Regulator, IVR)調整成 V_{HIGH} 、操作頻率調整為兩倍的時脈頻率(2x clock)，同時將其餘的區塊的操作電壓調整為 V_{LOW} 、頻率維持在原始頻率(1x clock)。另外，這裡的 IVR 也支援 near-threshold

voltage(NTV)，可以在短暫的 stall 時期中，將 graphics register file 及 ROM arrays 達到 NTV operation，也就是說在不影響功能的情況下，盡可能地降低操作電壓，進而降低整體的功率消耗。

要在 EU 之中實現這種 fine-grain DVFS 機制需要在原始的 EU 之中加上不同的電路來支援，整體修改後的 EU 架構圖如圖 3-6。在 fine-grain DVFS 中，調整電壓或是調整頻率個別需要不同的電路。調整電壓方面，作者設計了一個混合 DLDO/SCVR 的數位控制 IVR(digitally controlled hybrid DLDO/SCVR)，這個 IVR 的特性是高轉換效率(conversion efficiency)以及提供大範圍的電壓輸出，並且讓 IVR 可以快速的切換電壓、支援電源閘控以及功率代價(power overhead)盡可能地減少。在調整頻率方面，則是需要額外的時脈抗突波(glitch-less)除頻器/多工器(clock divider/mux)以及同步電路(synchronization logic)，讓 EU 可以有不同頻率的來源選擇以及讓 EU 在往 1x 時脈的區塊傳輸資料功能上不會出錯。所以在 EU-turbo 的波形圖上也可以看出，原始的 EU 在 EU-intensive 時期只能將整體 GPU 頻率跟電壓一起往上作調整，在非 EU 區塊之中的功率消耗就是多餘的，而修改後的 EU 則是在 EU-intensive 時期只對於 EU 作頻率電壓調整，整體來說可以提升 GPU 的能量效率。

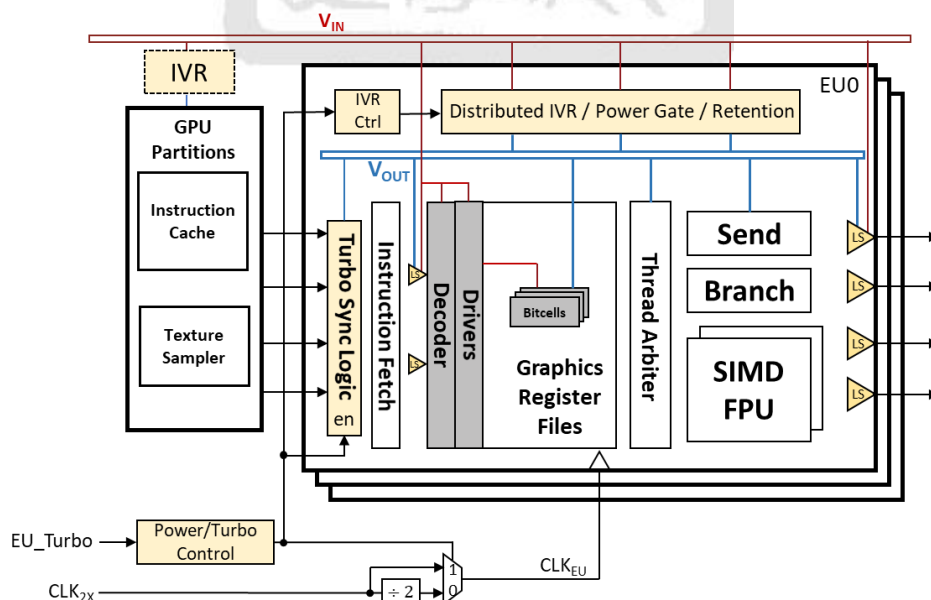


圖 3-6 Modified-EU 架構圖 [6]

3.2.2 Data-Traffic Aware DVFS

文獻[7]提出了一個考量動態硬體資料流量偵測(Data-Traffic Aware, DTA)的動態電壓頻率動態功率管理方式。整體系統架構如圖 3-7。

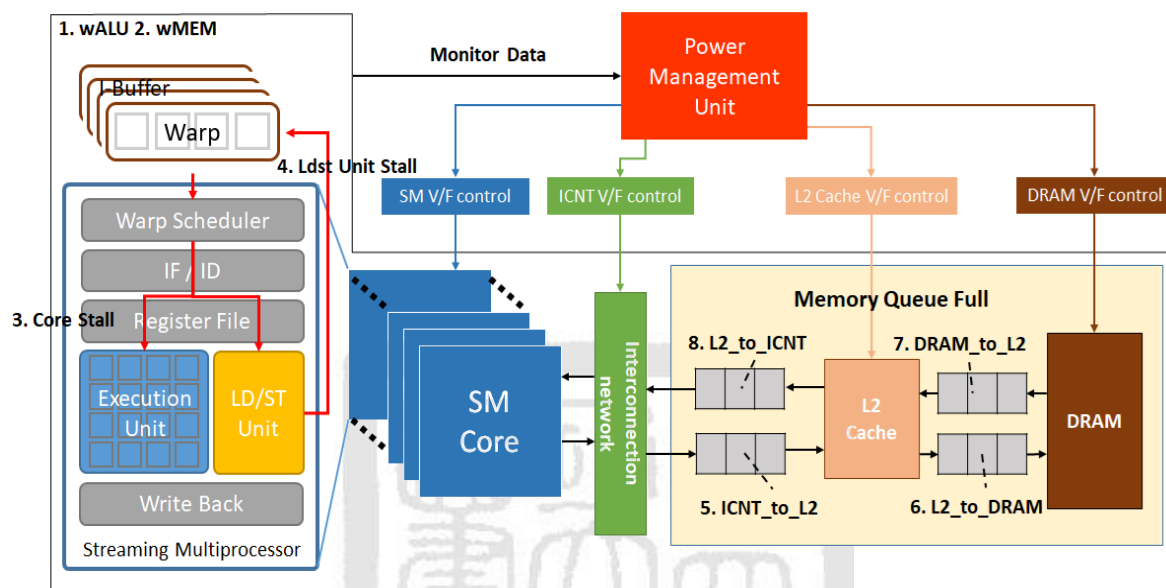


圖 3-7 DTA 系統架構圖 [7]

這篇論文主要是提出了一個在 GPGPU 運行期間觀測方法來進行 DVFS 的調整。在這篇論文所針對的 GPGPU 架構之中，可以切分為四塊時脈區塊(clock domains)，而在 GPGPU 運行期間觀測出的資訊就可以分別對這四塊區域作出頻率電壓的調整。首先作者在 GPGPU 中安插了各個監測點(monitor)，其中四個在 SM Core 內部，分別是 wALU、wMEM、Core Stall 以及 Ldst Unit Stall，而另外的四個則是位於記憶體區塊，分別是 ICNT_to_L2、L2_to_DRAM、DRAM_to_L2 和 L2_to_ICNT。在 SM Core 中的四個監測點，wALU 以及 wMEM 分別是用來計算在一段時間間隔之中，因為管線停滯而無法送入 ALU 和 MEM 的 Warp 數量，這兩個監測點是用來觀測目前的 Kernel 是運算單元密集的情況或是記憶體存取密集的情況。而 Core Stall 以及 Ldst

Unit Stall 則是進一步紀錄每個時間間隔中，SM Core 的管線停滯以及記憶體存取單元的停滯，藉此觀測電壓頻率的調整是否到位。

而在記憶體部分的四個監測點則是分別安插在不同階層的記憶體，藉由在記憶體之間佇列(queue)的壅塞程度來決定 DVFS 的調整，如同圖 3-8 所示。在偵測到佇列有壅塞的情況時，會將 Push 端(Clock Domain A)的頻率降低或是提升 Pop 端(Clock Domain B)的頻率，藉此提升整體的能量效率。

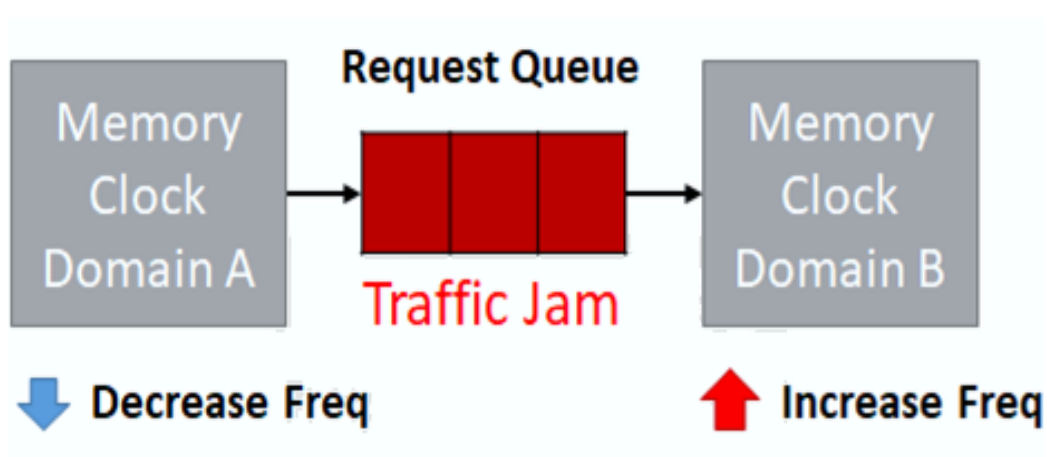


圖 3-8 佇列壅塞情況下的頻率調整方式 [7]

3.3 相關文獻總結

這個章節之中，整理了三篇對於圖形處理器的相關文獻，三篇的目標都是要提高整體圖形處理器的能量效率，而個別對於圖形處理器提出了不同的低功耗技術以及功率管理方法，在這個小節之中將綜合以上幾篇論文的觀點，整理如表 3-1，並且以下列幾點作為總結。

表 3-1 相關文獻總結

	Related Work [4]	Related Work [6]	Related Work [7]
Publication	JSSC, 2012	ISSCC, 2018	ISCAS, 2019
Target	Mobile CPU and GPU	GPU	GPGPU
Evaluation	Chip	Chip	GPGPU-Sim
Low Power Technique	Integrated two dies DVFS (V/F optimize) V_t swapping Power gating ...	DVFS Retentive Sleep Power gating	DVFS
DVFS control	Not mentioned	Fine-grain DVFS on EU	Monitor-based
Main idea	Implement several low power technique, forming a large SoC	EU turbo, fine-grain DVFS on key power/performance critical blocks	Set monitors to detect data traffic, making DVFS decision according to the result

在三篇文獻中都有採用動態電壓與頻率調整(DVFS)作為提升能量效率的方法，不過在使用上有很大的差異。在文獻[4]中，雖然提出了一個完整的低功耗技術的系統設計，但是在 DVFS 的部分，只有提到透過 DVFS 的技術來針對個別操作電壓點上的操作頻率進行優化，並沒有提出在運行過程中如何控制 DVFS 的方法。而在文獻[6]的部分，對於通用計算圖形處理器內部(就是其中的 Execution Unit)有細緻的 DVFS 控制，但在文獻中的方法也有一些問題存在，那就是這種方法必須有額外的許多電路來達成這種細緻的控制，在設計的複雜度方面以及硬體付出的代價都是比較大的；另一方面，在這篇文獻中主要是針對 EU-dominant 的狀況來考慮，這樣對於 EU-dominant

的狀況可以得到不錯的好處，但是這樣在 EU 使用沒那麼密集的狀況或是記憶體使用密集的狀況都會得到比原始 EU 更差的結果。所以如果把文獻[6]中的通用計算圖形處理器的設計情境轉移到邊緣運算裝置的話，必須要思考的是額外的電路所造成的硬體代價是否會對整體的系統有好處，以及如何在通用計算圖形處理器運行過程中判斷當前的運作趨勢為運算密集或者是記憶體存取密集的狀況，再來判斷是否需要對電壓與頻率進行調整。在文獻[7]之中則是利用安插觀測點在通用計算圖形處理器之中來判斷當前的運作趨勢，進而對於不同的時脈區塊作頻率電壓的調整。這種方法的缺點在於文獻中所抓出的觀測點通常是針對平台中(在這篇文獻之中是使用 GPGPU-Sim[9] 模擬器)的特定資料傳輸訊號線所產生的觀測點，在平台變更或是更換不同的架構後，再次使用同一套方法就會有觀測點或是效果上的差異；另一方面，這樣觀測點方法是間接的從通用計算圖形處理器內部各處蒐集運行的狀況進而判斷運作趨勢，若能有一個更直接明確的硬體資訊，對運作趨勢的判斷就能更精確，如果有新架構或是平台變更，兼容性也可以更好。

總結來說，提高能量效率最主要的目標還是關鍵的地方可以高功耗，也就是可以讓關鍵區塊藉由提高電壓頻率來加速運作，而非關鍵區域就要藉由關停、降低電壓頻率等方式，盡可能的節省功率的消耗。而如果要動態進行功率管理，最需要考量的就是如何在動態運行間判斷現在的應用程式所需要的關鍵區塊是在何處。而由於通用計算圖形處理器內部架構龐大且運行的資料流複雜，如果能找到更直接有效的硬體資訊來做關鍵區域的判斷，對於整體功率管理的效果也能有所提升。

第 4 章 硬體動態使用率電壓及頻率的動態 功率管理

在這個章節裡，首先會在 4.1 節描述本論文所要解決的問題；接著在 4.2 節之中會介紹在本論文中使用的平台之環境與架構，其中包含通用計算圖形處理器的模擬平台介紹以及功率模型的建置等；最後於 4.3 節中詳細介紹本論文所提出的硬體動態使用率功率管理方法，其中包含了低功耗技術的使用解說以及動態功率管理的演算法細節等。

4.1 問題描述

就如同在 1.1.1 節中所提到的，邊緣運算的應用及市場近年來快速地成長，邊緣運算裝置的設計與開發也逐漸受到關注。邊緣運算裝置和一般的裝置不同，在電源來源及體積限制之下，邊緣運算裝置不再往高運算速度的方向發展，因為高運算速度的裝置通常也意味著可觀的功率消耗。因此，在設計邊緣運算裝置中的通用計算圖形處理器時，需要考量的就是如何能在最少的功率消耗之中，得到最好的運算效能，也就是以提高整體的能量效率為目標。本論文提出適用於邊緣運算裝置的高能量效率通用計算圖形處理器功率管理機制，設計上考量了硬體的動態使用率來進行功率管理並且同時使用了時脈閘控的低功耗技術來降低整體的功耗。而功率管理目標是要找出當前運行之中效能關鍵的區塊，透過提升關鍵區塊的電壓頻率或者降低非關鍵區塊的電壓頻率，在不降低關鍵效能的情況下盡量減少其他區塊的功率消耗，而達到高能量效率的目標。

4.2 目標平台之環境與架構

目標平台為由成功大學計算機架構與系統實驗室(CASLAB)開發之 CASLAB-GPUSIM[2][10][11][12]，本章節將針對該平台做介紹，包含了平台介紹、內部硬體 (SIMT Core)架構介紹以及功率模型的說明。

4.2.1 平台介紹

CASLAB-GPUSIM 是一個 GPGPU 的運算模擬平台，整體平台的系統架構圖如圖 4-1 所示。

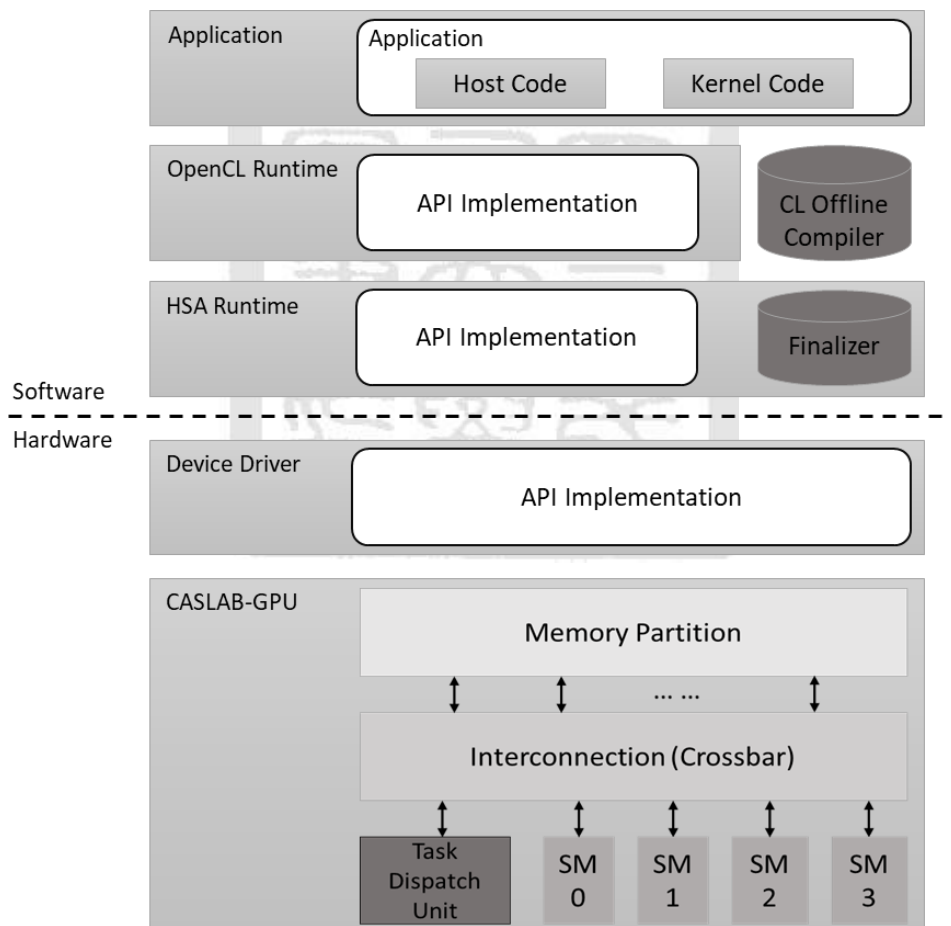


圖 4-1 CASLAB-GPUSIM 平台架構圖 [13]

CASLAB-GPUSIM 是針對 HSA 組織所訂定的規範而開發出的平台，可以支援 HSAIL(Heterogeneous System Architecture Intermediate Language)這種語言[14]，該語言最主要的概念在意圖簡化程式設計模式，進而使軟體開發人員可以更有效的運用 CPU 及 GPU 的功能，示意圖如圖 4-2，也就是說，對於軟體程式設計師或是編譯器開發者而言，只需要著重在如何將 C/C++、OpenCL 這類的高階語言轉換成 HSAIL 即可。在平台軟體層主要是由自行開發之 OpenCL Runtime 以及 HSA Runtime 函式庫組成，其中包含了 CL Offline Compiler 以及 Finalizer。CL Offline Compiler 是用來將 OpenCL 的 Kernel 程式轉換成 HSAIL 的形式，再透過 Finalizer 將 HSAIL 轉換為平台的自訂指令集，讓平台可以執行 HSAIL 指令集。而在硬體層的部分主要是由 SystemC 語言所實作的通用計算圖形處理器硬體，整體的硬體架構與 2.1 節中所描述的行為類似，Task Dispatch Unit 在接收軟體層處理的封包後，在發配任務到底層的 SM Core 執行，並透過 Crossbar 與 DRAM 進行資料的交換。

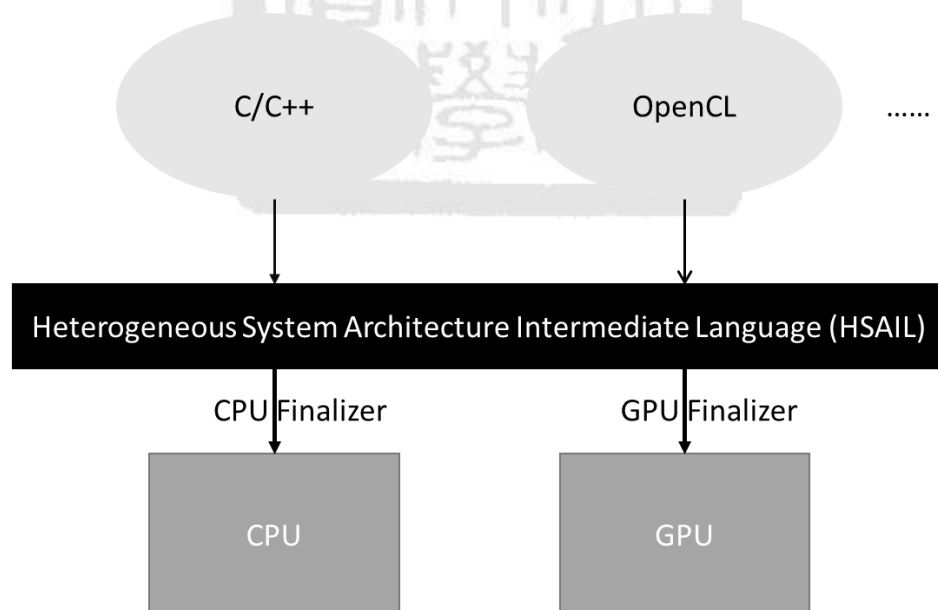


圖 4-2 HSAIL 示意圖

4.2.2 多重串流處理器內部架構

在圖 4-3 中展示了 SM 的內部架構，以下會針對內部各個元件做一個簡介。

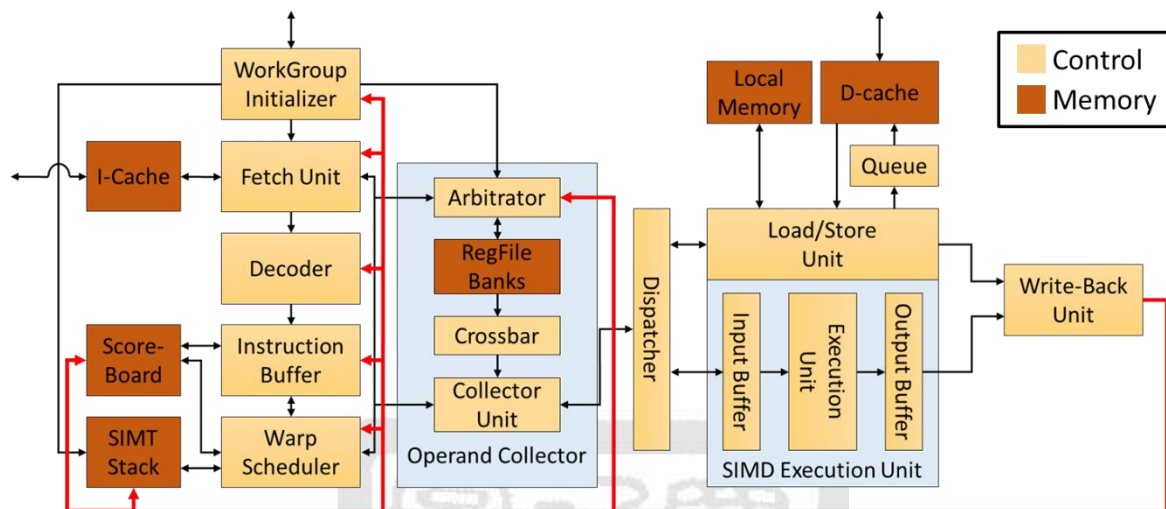


圖 4-3 多重串流處理器架構

Fetch Unit 為 SM 內部負責抓取指令的元件，Fetch Unit 中有 48 個 Warp 的 Program Counter (PC)，在 WorkGroup Initializer 初始化之後會初始化各個 Warp 的 PC，接著在將選定的 PC 傳到 I-Cache 中抓取指令，再把抓到的指令傳至 **Decoder** 做指令的解碼。

Instruction Buffer 是指令緩存單元，收到來自 Decoder 的指令與 PC 之後，會將各個 Warp 存放至 Instruction queue 中，接著會順序檢查在 queue 之中的各個指令是否可能被派發，派發成功後會將其移出 Instruction queue。

Scoreboard 是用來記錄和檢查暫存器相依性(dependency)的元件，主要是用來避免 Read-After-Write(RAW)以及 Write-After-Write(WAW)的資料危障(Data Hazard)情況，當 Warp 被發派到後端執行時，該指令的 destination register 會被記錄在 Scoreboard 之中，後續有其他指令需要使用暫存器時，Instruction Buffer 會先跟 Scoreboard 確認相依性，若有被存在 Scoreboard 之中則該指令無法成功派發，而被記錄在 Scoreboard 之中的暫存器會在資料被寫回時被釋放。

SIMT Stack 為存放每個 Warp 的 SIMD 遮罩(SIMD Mask)的元件，其功能在於處理分支分歧(Branch Divergence)問題，其中會存放 PC、reconvergence PC 以及遮罩，遮罩是用於判斷 thread 的寫回是否有效的根據。

Warp Scheduler 是負責 Warp 派發的單元，透過不同的機制如 LRR、GTO 等將指令派發給後端，Warp Scheduler 在派發指令前，先確認 Warp 的 Stall 資訊以作為能否派發的資訊，再將指令與 Scoreboard 做相依性檢查，確認可以派發之後，再從 I-Buffer 與 SIMT Stack 中取出指令相關資料。

在 **Operand Collector** 中含有四個元件，分別是 **Collector Unit**、**Collector Arbitrator**、**Collector Crossbar** 及 **Register File**，這一整塊的 Operand Collector 最主要的目的就在從 Register File 中蒐集指令所用到的運算子而後傳給後端管線做處理。在 Warp Scheduler 派發指令之後，Collector Unit 會負責緩存所有 threads 的 source operand。而 Collector Arbitrator 則是負責處理所有需要存取 Register File 的請求，在每一個 cycle 之中都會挑出一個存取請求(access request)去向 Register File 做讀或寫的動作。

在後端管線之中主要就是由 **Execution Unit** 以及 **Load Store Unit** 處理傳到後端的指令，Execution Unit 會負責將收到的指令根據指令集算出相對應的結果，而 Load Store Unit 則是負責對記憶體讀寫存取請求的相關指令。在算出結果或是從記憶體讀取出資料之後，就會透過 **Write Back Unit** 將會做 commit，將資料寫回。

4.2.3 功率模型

在對通用計算圖形處理器平台進行各類型的功率消耗優化之前，需要先針對當前的平台建立一套功率模型(Power Model)，才可以了解在進行功率管理後對於整體功率消耗的影響有多少。在建立功率模型方面，在架構的設定確定的情況下，本實驗室(LPHP-LAB)根據記憶體類型以及非記憶體類的元件有不同的建立模型方式。根據記憶體類型的元件如 Register File、L1 I-cache、L1 D-cache 以及 L2 Cache 使用 ARM Memory Compiler 於 TSMC 40nm 製程所產生對應的 RegFile 以及 SRAM 硬體，並將個別的讀取功率、寫入功率以及待機功率的功率數值整合進功率模型，在應用程式運行於 GPGPU 平台時就會依據不同的記憶體存取行為進行功率的計算。而在 DRAM 的部分則是參考 Micron Technology, Inc.的 DDR3L 規格書[15]，將相對應之功率資訊整合於平台之中。而非記憶體類元件的部分則是透過底層的硬體實作而取得功率資訊，其功率資訊取得流程如圖 4-4，流程說明如下。

1. 針對個別在 SystemC 平台中的模組(為了說明簡便，簡稱為 module α)，在平台運行應用程式(benchmark)時，透過寫波形檔(.vcd)的方式萃取出 α 的輸入輸出訊號在各個時間點的值，再透過 Python 語言將波形檔中的輸入輸出訊號轉換成 Verilog 的可以使用的測試訊號(stimulus)。
2. 在 Verilog 中實作 α' ，行為及時序需與 SystemC 平台相符。透過在步驟 1 中取得的 α 輸入測試訊號傳進 α' ，得到 α' 的輸出訊號之後再與步驟 1 中得到的 α 輸出測試訊號比對，若有不同之處，則修改 α' 的行為，重複這個步驟直到確認 α 與 α' 的行為相同為止。
3. 將完成的 α' 使用 Synopsys Design Compiler，在標準元件庫 TSMC 40nm 製程之下進行合成，得到的 gate-level 模組再跑一次 benchmark，將得到的波形檔使用 Synopsys PrimeTime PX 進行功耗的分析。

4. 將運行六隻應用程式所得到的平均功率資訊整合進功率模型之中，整體功耗誤差如圖 4-5。

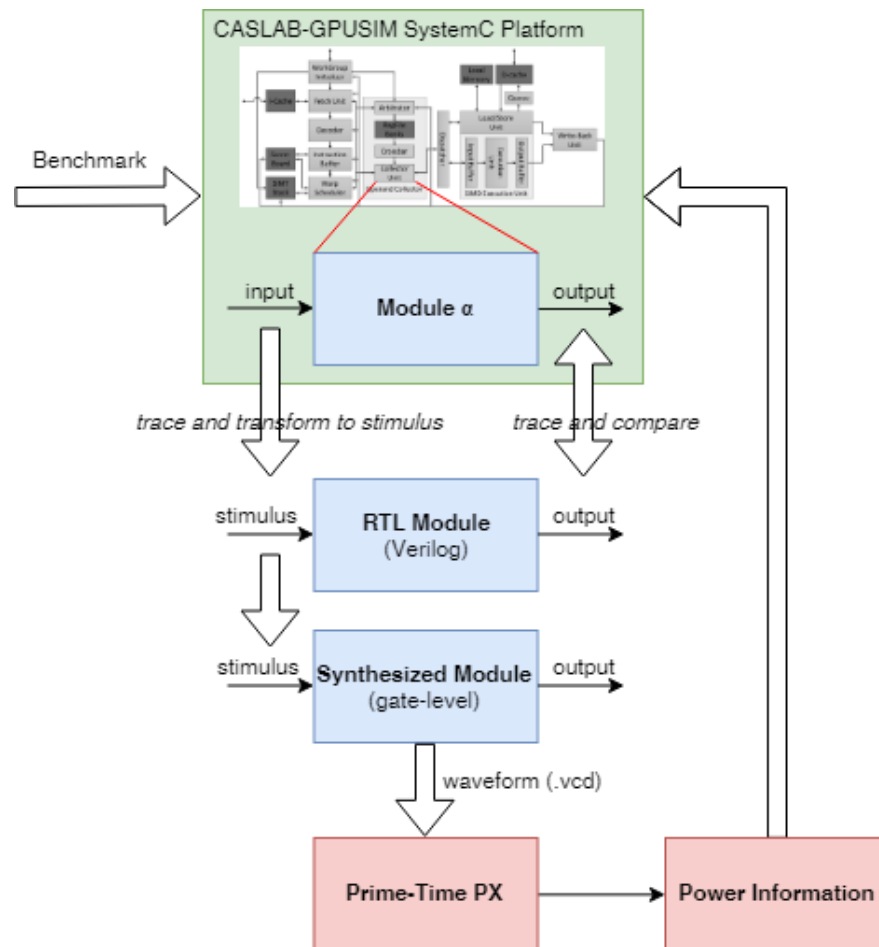


圖 4-4 非記憶體元件功率資訊取得流程圖

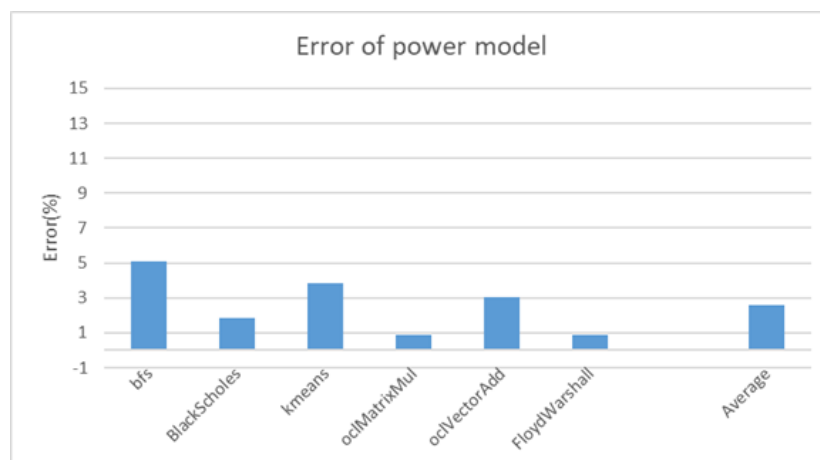


圖 4-5 非記憶體元件功率模型誤差

4.3 低功耗技術以及功率管理機制

本論文使用 CASLAB-GPUSIM 之通用計算圖形處理器與本實驗發展的硬體功率模型為平台，提出低功耗技術的設計以及考量硬體動態使用率之功率管理方法，提升整體通用計算圖形處理器之能量效率。

4.3.1 低功耗技術的設計

在整個通用計算圖形處理器中的運算核心 SIMT Core 中使用了大量的暫存器(register, 在數位電路之中通常是使用 D Flip-Flop (DFF)做為暫存器)來儲存資料。在前端的部分需要暫存許多 Warps 的資訊，包含在 I-Buffer 之中需要在 Instruction queue 中緩存各個指令資訊和在 SIMT Stack 中使用大量的暫存器來儲存 PC、reconvergence PC 以及遮罩資訊等。在後端則是有 Collector Unit 緩存 threads 的運算子以及在 Write Back 元件中需要儲存要被寫回的資料等等。在有這麼龐大數量的暫存器的情況之下，當時脈(clock)開始運作，clock 開始傳入 D Flip-Flop 之中就會消耗大量的動態功率。只要時脈持續的在震盪，當運算核心進入閒置狀態，動態功率仍然會持續的消耗。根據這個情形，本論文提出了在模組層級的時脈閘控(module-level clock gating)設計，以最少的硬體代價，盡可能換得最多的動態功耗降低。

圖 4-6 是一個一般的數位電路架構圖，通用計算圖形處理器中運算核心的電路架構也是如此。一般來說，電路設計分成組合電路(combinational circuit)以及序向電路(sequential circuit)，以當前的數位電路來說，序向電路通常是由 DFF 做為暫存器所組成。組合電路會一直進行運算，並且透過一些致能訊號(EN)來決定是否會將運算結果傳至後級。當 clock 有跳動的行為(正緣或是負緣觸發)時，DFF 就會將運算的值儲存以做為下一級的組合電路使用。

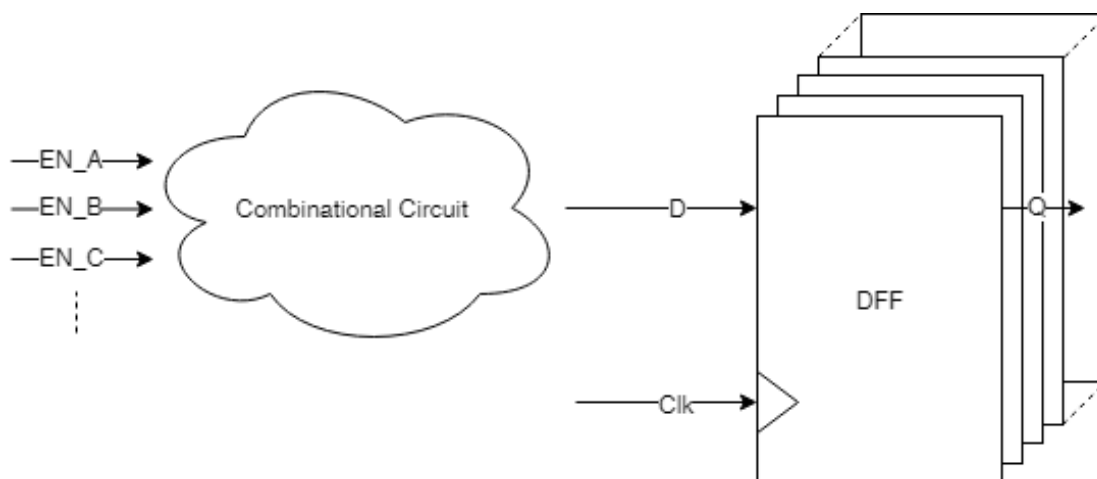


圖 4-6 一般數位電路架構圖

加入時脈閘控設計的架構圖如圖 4-7。時脈閘控的設計需要先了解 clock 可以被關閉(gating)的時機點，本論文的设计方式是將所有組合電路中的致能訊號另外傳至一個或閘(OR gate)中以做為 clock gating cell 開啟與否的依據。在此處或閘的用意在於確認當前組合電路是否進行有效的運算，在有任一致能訊號拉起的狀況下，clock gating cell 的致能訊號為 1，表示 clock 會正常進入 DFF 之中儲存組合電路所運算的結果，而當所有致能訊號都為 0 之時，表示組合電路並沒有要更新在 DFF 內的值，此時就會將 clock gating cell 的致能訊號拉為 0，關閉 clock 的震盪，降低整體模塊的動態功耗。

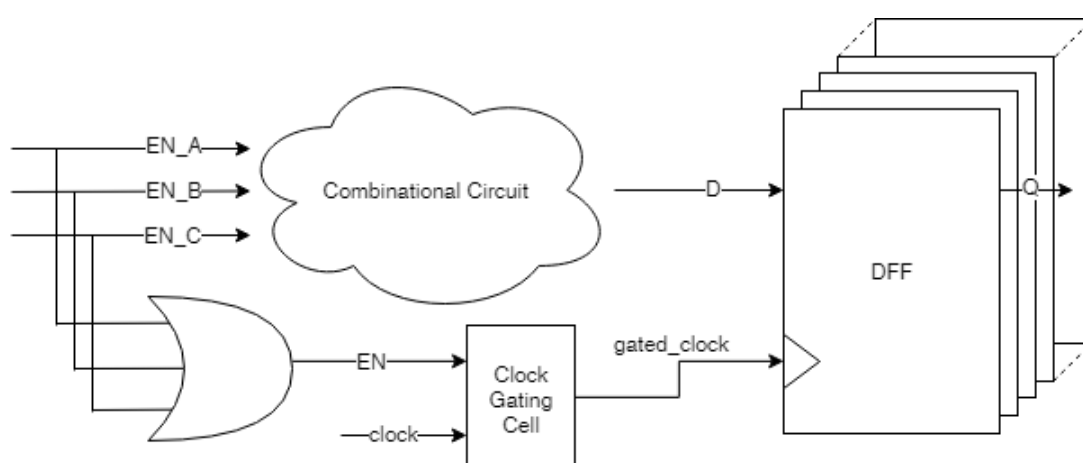


圖 4-7 加入時脈閘控設計之電路架構圖

而對於時脈閘控的目標模組選擇，本論文分別將上述的時脈閘控設計使用於 SIMT Stack、Collector Unit 以及 Write Back 之中，這三個元件於 SIMT Core 中的位置如圖 4-8。

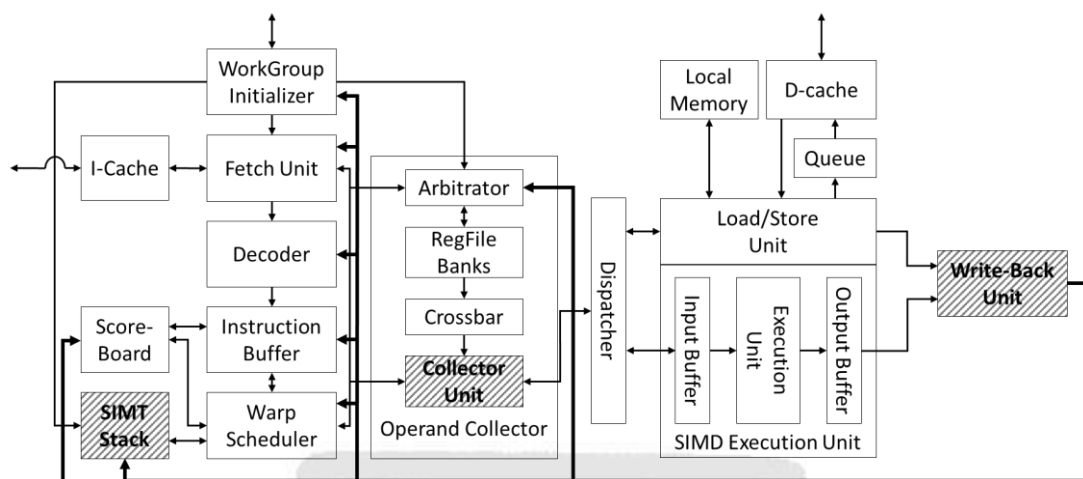


圖 4-8 使用時脈閘控設計之模組

選擇這些模組最主要的原因是，在當前平台之中，這三個模組的功耗佔整顆運算核心功耗的一半以上，且其中以動態功耗為主，功耗分布圖如圖 4-9。在這樣的條件之下，採用時脈閘控的設計於其中可以用最少的硬體付出使動態功耗有顯著的降低。另一方面，這三者 SIMT Core 中的分布分別位於前端、後端以及介於前後端之間 Warp 發配位置附近，在這三個模組之中使用時脈閘控的設計對於後續觀察整顆 SIMT Core 的使用狀況可以得到更好的效果。

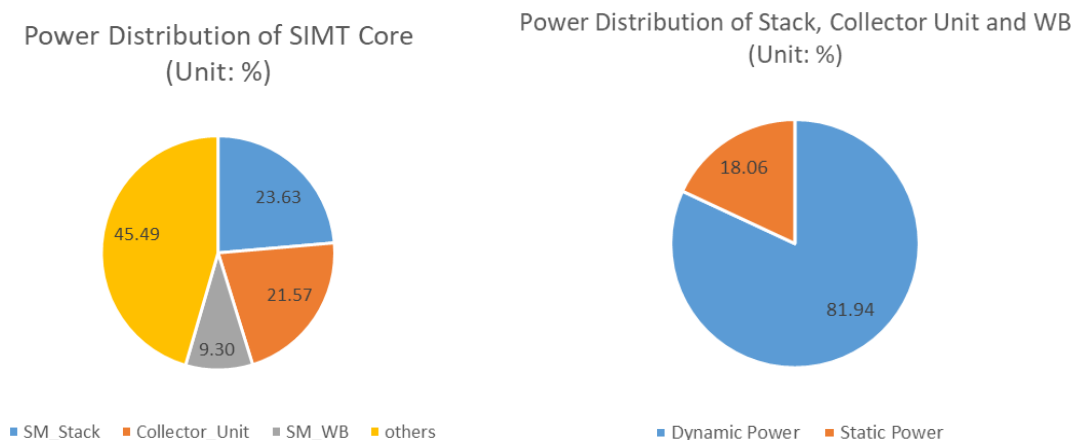


圖 4-9 功率消耗分布圖

4.3.2 硬體動態使用率偵測及電壓頻率調整

本論文依據上述設計於各個模組之中實現了時脈閘控技術，在這個基礎之上，透過當前的一段時間之中時脈開或關的比例，可以在程式運行中(runtime)了解當前應用程式是運算核心使用密集(Core-intensive)或是記憶體存取密集(Memory-intensive)的趨勢。由於在時脈閘控的設計之中，時脈開或關是依據各模組的致能訊號有沒有拉起而決定的，所以當時脈關閉的時間比例較高時，代表著在當前狀態之中，運算核心是處於閒置的狀態或是在等待記憶體存取以致於時脈關閉的時間較長；反之，當時脈開啟的時間比例較高時，表示較少等待記憶體存取的時間或是在運算核心之中較少因為各類因素造成管線停滯的時間，也就是說在當前的狀況之中整體運算核心的使用是較為密集的。根據上述的情況，對於運算核心使用率(utilization)有了以下的定義：

$$utilization = module\ usage = \frac{clock\ cycle\ count\ (gated\ clock)}{clock\ cycle\ count\ (clock)} \quad (4.1)$$

以圖 4-10 的波形及電路圖做為說明。在模組之中如果致能訊號並非皆為 0 的狀況之下，Clock Gating Cell 的致能(EN)訊號就會被拉起，使 DFF 有接收到震盪的 gated clock，而若模組之中致能訊號皆為 0，Clock Gating Cell 的致能訊號就會被關閉，將 gated clock 都拉成 0。依照上方本篇論文對於使用率的定義，在這張波形圖之中，gated clock 的 cycle count 為 3、clock 的 cycle count 為 5，因此使用率為 0.6。

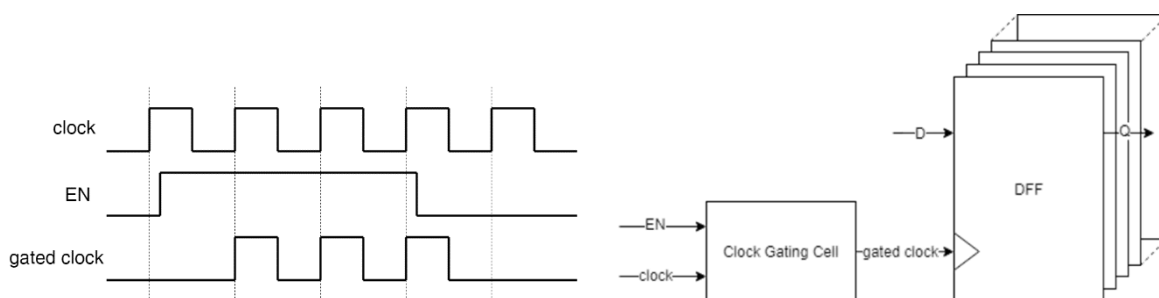


圖 4-10 使用率定義之說明圖

為了避免單一模組可能因為前後級管線遲滯或是其他問題而造成使用率異常的狀況，本論文參考了分別位於運算核心的前級、後級級前後級之間的模組(也就是 Stack、Collector Unit 及 Write Back)的使用率來做為整顆運算核心使用率的參考。

在本論文所使用之 CASLAB-GPUSIM 平台中的通用計算圖形處理器有分為兩個時脈區塊(clock domain)，如圖 4-11 所示。在平台中主要分成運算核心的 SM Core 頻率以及記憶體區塊的 DRAM 頻率，在平台上的 L2 Cache 頻率是與 DRAM 頻率在同一個時脈區塊，而 Interconnect 的部分則是以 SystemC 中的 TLM 方法來做 SM Core 與 DRAM 之間的資料交換，目前並沒有 Interconnect 的時脈區塊。

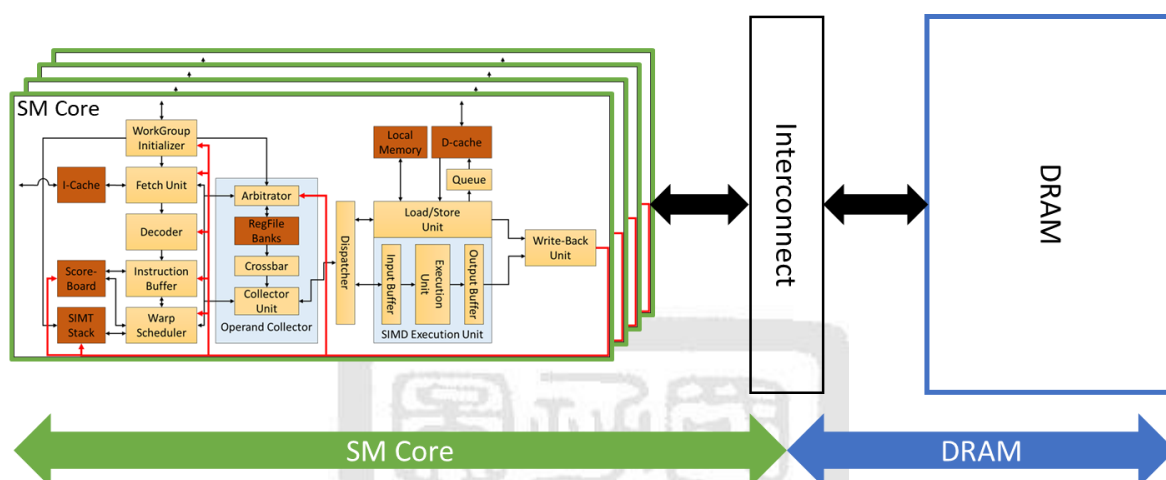


圖 4-11 CASLAB-GPUSIM 時脈區塊

在時脈區塊被切分為兩塊的情況下，對於功率管理而言，要提升整體通用計算圖形處理器的能量效率最主要的目標就是要找出當前應用程式的運行趨勢是屬於運算核心使用密集(core-intensive)或是記憶體存取密集(memory-intensive)的狀況。因為就如同本論文前面提的概念，要達到高能量效率或是低功耗的目標，就是要降低非關鍵區塊的功率消耗，並且視情況提升關鍵區域的電壓頻率，提升整體的能量效率。因此，本論文使用了前面定義之硬體運算核心使用率來進行功率管理，當運算核心使用率高時，則將當前的運行狀況判定為運算核心使用密集，運算核心使用率低時，則判定為記憶體存取密集。接著再依據判定的趨勢對於兩個時脈區塊來進行電壓與頻率調整，整體電壓頻率調整之功率管理演算法流程如圖 4-12。

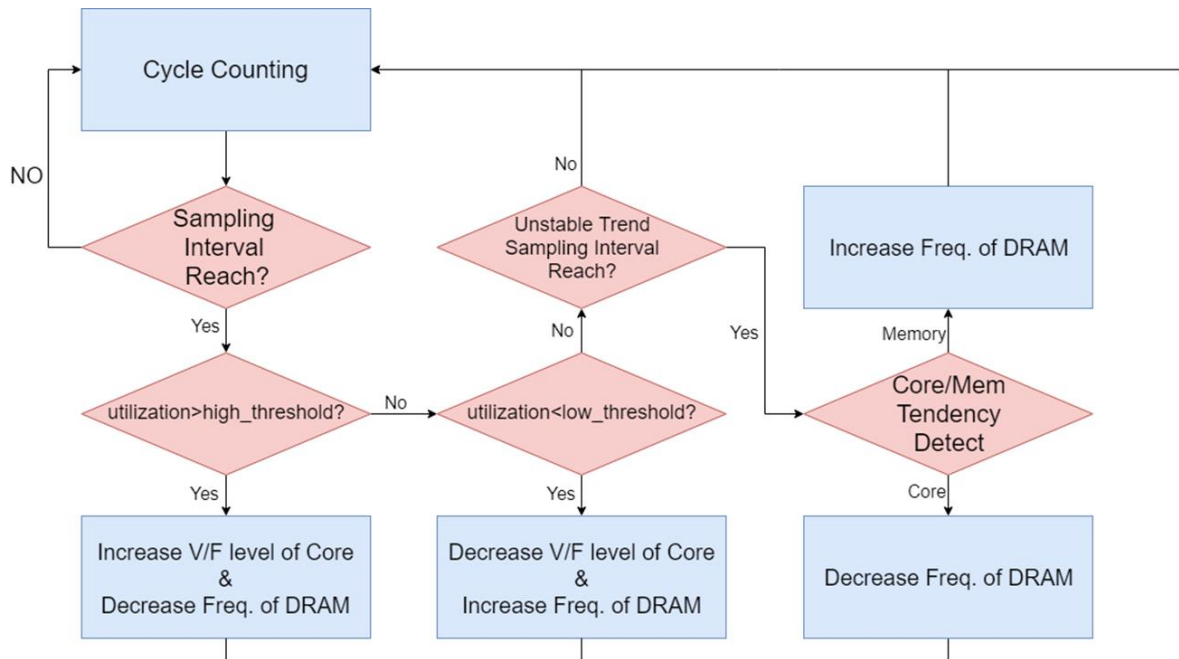


圖 4-12 硬體動態使用率功率管理演算法流程

演算法會在程式執行中(runtime)持續蒐集硬體使用率資訊，每隔一段取樣時間間隔(Sampling Interval)會判斷當前取樣時間間隔的運作趨勢。當運算核心使用率高於 high threshold 時，就表示是運算核心使用密集的狀況，此時演算法會調升運算核心這個關鍵區塊的電壓頻率，並同時調降非關鍵的記憶體區塊的頻率；反之，當運算核心使用率低於 low threshold 時，運作趨勢為記憶體存取密集，此時則是調升記憶體區塊頻率而同時調降運算核心的電壓頻率。另外，有一種狀況是當前的運作趨勢不夠明確的情況，本論文提出另一套方案來進一步判斷趨勢。在這個方法之中，會在一段趨勢判定時間間隔(Unstable Trend Sampling Interval)之中，將在 Instruction Buffer 之中的所有指令分為運算用的指令或是需要存取記憶體的指令並計算個別的指令數量，接著在兩段趨勢判定時間間隔之中計算當前運算指令及記憶體指令的數量與上一段趨勢判定時間間隔的數量變動率，在此變動率的計算為當前指令數量與上一段時間間隔指令數量的比值，如式(4.2)。

$$instruction\ change\ rate_{type} = \frac{instruction\ count_{type}(current\ sampling\ interval)}{instruction\ count_{type}(last\ sampling\ interval)} \quad (4.2)$$

當運算指令之變動率大於記憶體指令之變動率時，可以預測未來一段時間運算核心的使用較為密集，因此在此處會調降記憶體區塊的頻率來降低非關鍵區塊的功率消耗；反之，若運算指令之變動率小於記憶體指令之變動率，則認定為記憶體存取密集的趨勢而調升記憶體區塊的頻率。

另外，為了避免過度的電壓頻率調整以及讓演算法在不同應用程式或不同架構之中有更好的擴充性，本論文提出了一個有限狀態機(Finite State Machine, FSM)來進一步優化功率管理。這個狀態機會利用狀態之間的轉移來避免因為異常的使用率狀況而做出的電壓頻率調整，以及可以進行使用率閾值(threshold)的調控，讓閾值可以隨著不同的運行狀態而對使用率高低有不同的判斷，讓功率管理演算法更能適應各類不同的情況，整體 FSM 狀態圖及狀態轉移條件如圖 4-13。

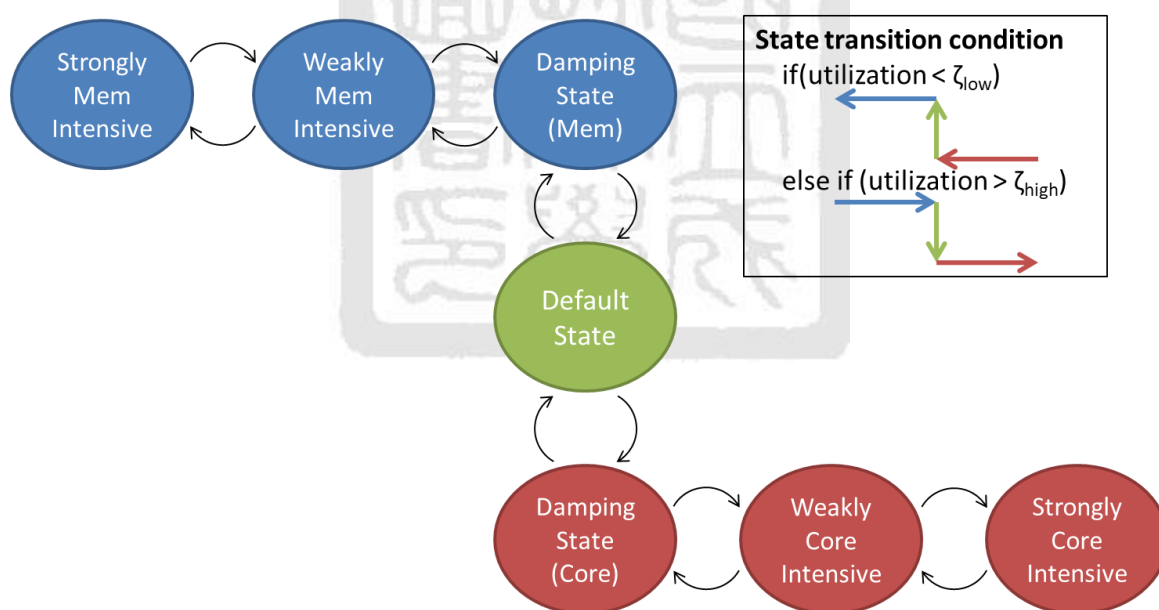


圖 4-13 FSM 狀態圖及狀態轉移條件

在這個 FSM 之中，初始狀態為 Default State，其餘狀態主要可以分為與 Core 和 Memory 相關的狀態，分別是 Damping State (Mem/Core)、Weakly Mem/Core State 以及 Strongly Mem/Core State。而狀態轉移條件則是依據使用率而決定，當經過一段取

樣時間間隔(Sampling Interval)蒐集完使用率資訊之後，若使用率小於低閾值時，狀態會由 Strongly Core State 往 Strongly Mem State 的方向轉移，當使用率大於高閾值時則方向相反，而當使用率介於低閾值及高閾值之間時，則狀態會往 Default State 的方向轉移。

Damping State (Mem/Core)用以防止電壓頻率過度調整，通用計算圖形處理器運行過程複雜，在運行期間偶爾會出現使用率大幅提升或下降的情況，所以當演算法判定目前使用率為低/高時會進入 Damping State (Mem/Core)，而進入 Damping State 時電壓頻率還不會進行調整。在連續兩段取樣時間間隔使用率都為低/高時，代表著趨勢確定為記憶體存取密集/運算核心使用密集的狀況，就會進入 Weakly Mem/Core State。而在這個 state 中就會開始進行電壓頻率的調整，同時也會將使用率閾值調降/調升，代表著演算法對於使用率低/高的判定會更嚴格，也是為了更進一步確認當前的趨勢是否為記憶體存取密集/運算核心使用密集。若在更嚴格的使用率閾值判定之下使用率仍為低/高時，此時就會進入 Strongly Mem/Core State，更進一步的調整電壓頻率值，並且再一次調降/調升閾值，代表著要維持在 Strongly Mem/Core State 的判定會更嚴格。總結來說，本論文提出的這個有限狀態機可以避免過度的電壓頻率調整以及在動態運行中調整使用率閾值，讓通用計算圖形處理器可以在遇到不同應用程式時有更多的調整空間。

第 5 章 實驗結果與分析

5.1 實驗環境設置

本論文使用 CASLAB-GPUSIM 作為通用計算圖形處理器的模擬平台，在平台中的硬體配置參數如表 5-1。

表 5-1 CASLAB-GPUSIM 實驗環境

CASLAB-GPUSIM	ver. PolyBench
ISA	HSAIL compatible
SM core number	4
Max # Warps	48
SIMD Lane Width	32
RegFile Size	1MB
L1 I-Cache Size	32KB
L1 D-Cache Size	16KB
DRAM	Micron DDR3L 1600

在本次論文中於 CASLAB-GPUSIM 中加入了動態電壓頻率調整的機制，在整體通用計算圖形處理器之中需要加上 PLL(Phase-Locked Loops)來調整 SM Core 及 DRAM 端的頻率、DC-DC converter 調整電壓值、一塊 PMU (Power Management Unit) 來管理整體頻率電壓的選擇。與功率管理相關之實驗環境設定與架構圖如表 5-2 及圖 5-1。

表 5-2 功率管理相關實驗環境設定

Power Management Related Setup	
SM clock rate (Default)	200 MHz
DRAM clock rate (Default)	200MHz
SM frequency level	200, 166.67, 100, 80, 66.67 MHz
DRAM frequency level	400, 200, 166.67, 100, 80, 66.67 MHz
DC-DC converter	0.9-0.72 V
Sampling Interval	10000 cycles
Unstable Trend Sampling Interval	40000 cycles
Utilization – high threshold	0.8 ± 0.05
Utilization – low threshold	0.5 ± 0.05

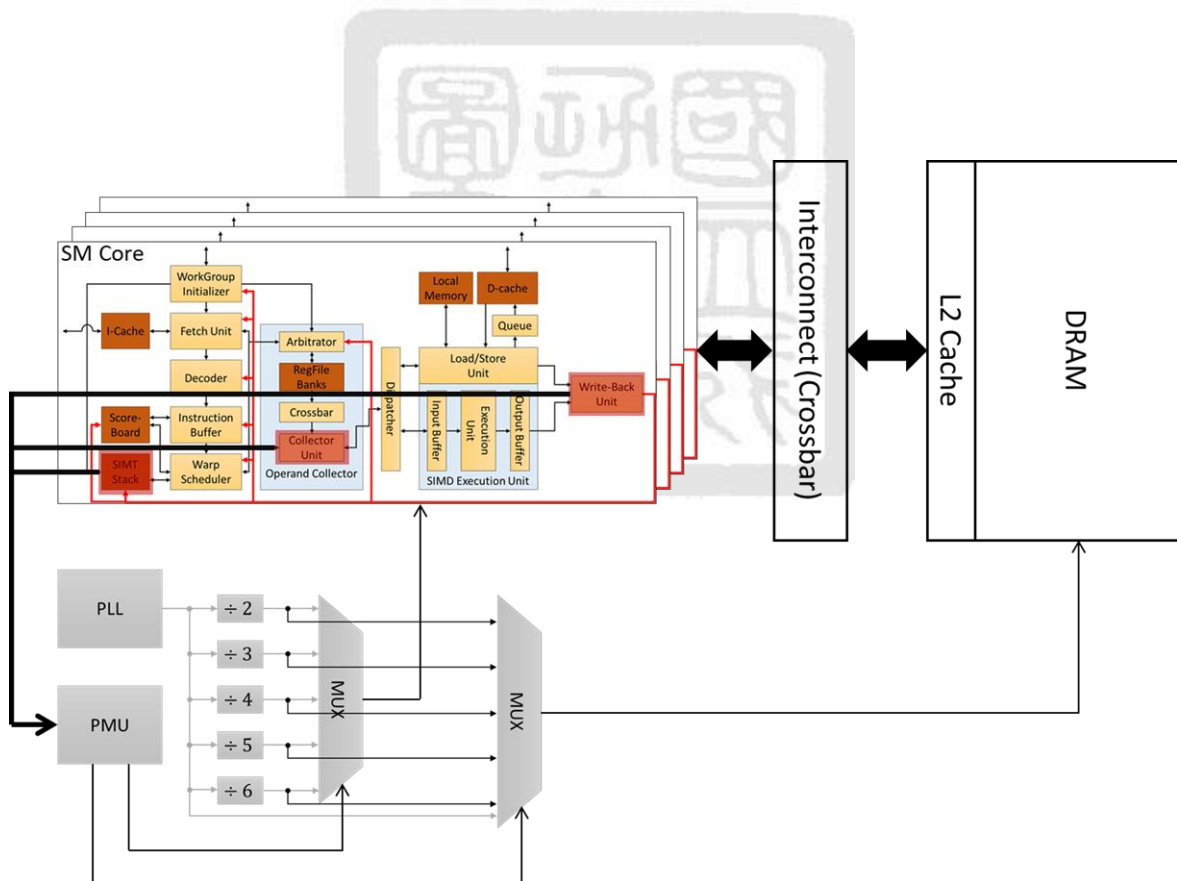


圖 5-1 功率管理架構示意圖

5.2 實驗結果

5.2.1 時脈閘控

在這個實驗中展示使用時脈閘控低功耗技術對於功耗之影響，圖 5-2 顯示時脈閘控於各模組中降低功耗的比例，而圖 5-3 則是有無時脈閘控對於整體通用計算圖形處理器功耗之影響。

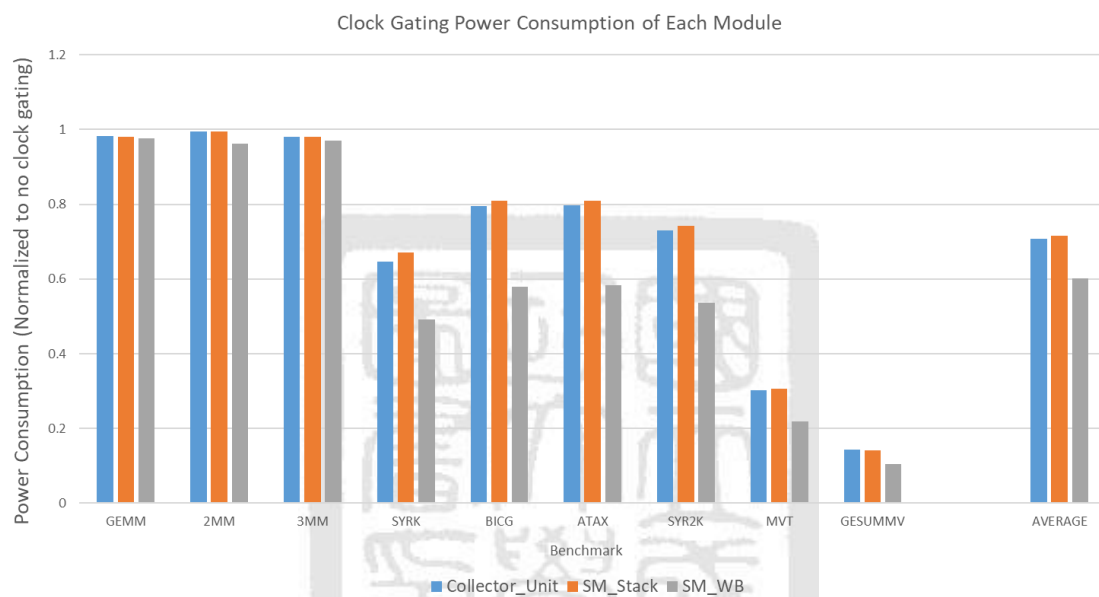


圖 5-2 時脈閘控於各模組中節省之功耗

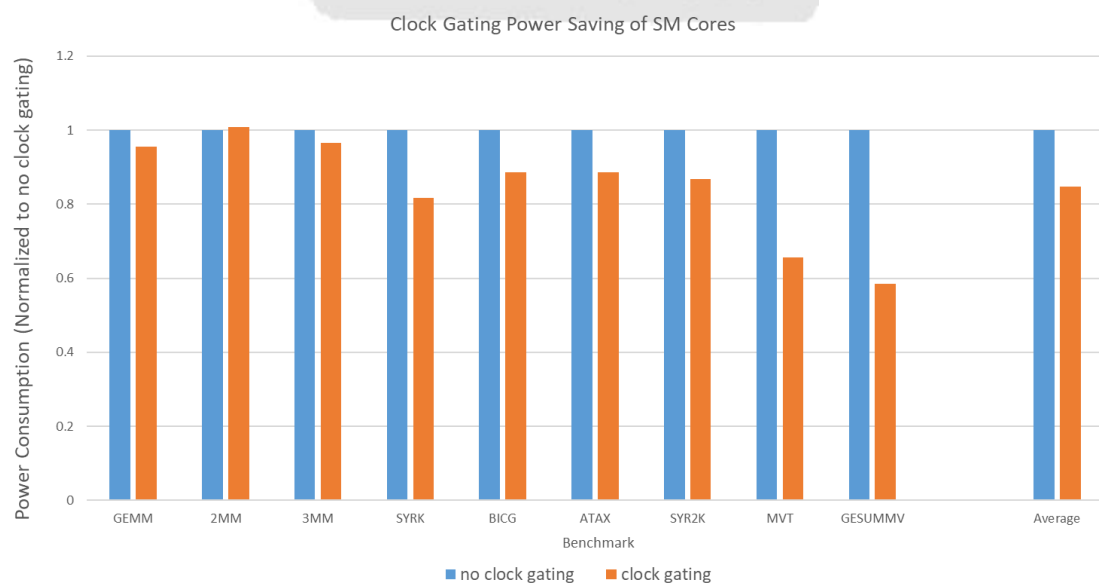


圖 5-3 時脈閘控於整體 SIMT Core 中節省之功耗

5.2.2 硬體動態使用率功率管理

在這個實驗之中對於硬體動態使用率功率管理演算法的結果進行分析。主要包含了演算法參數的決定、使用率閾值的選擇以及整體功率管理演算法的功率、效能及能量結果分析。

在本論文所提出之功率管理演算法中，取樣時間間隔(Sampling Interval)及趨勢判定時間間隔(Unstable Trend Sampling Interval)是影響功率管理效果的參數之一。在選擇時間間隔時，過短的時間間隔會有過度調整的問題，也就是取樣時間不夠長，看到的使用率只是片面的資訊，對於趨勢的判定不夠準確而做出的電壓頻率調整會有不符合當前主要趨勢的可能，而過長的時間間隔則是會有反應時間不夠快的問題，可能會有當前整體運作趨勢已經改變但電壓頻率值太慢做出調整的狀況發生。因此，對於取樣時間間隔及趨勢判定時間間隔的選擇，模擬結果如圖 5-4 及圖 5-5。

在圖 5-4 中展示取樣時間間隔為 5000、10000、15000 及 20000 cycles 的在運行 BICG 測試程式(Benchmark)的使用率結果，可以看到在 5000 cycles 時使用率有明顯的起伏的狀況，在判斷趨勢時就可能會失準，在時間間隔超過 10000 cycles 之後使用率就趨於穩定，所以在取樣時間間隔方面最後就是選擇 10000 cycles。

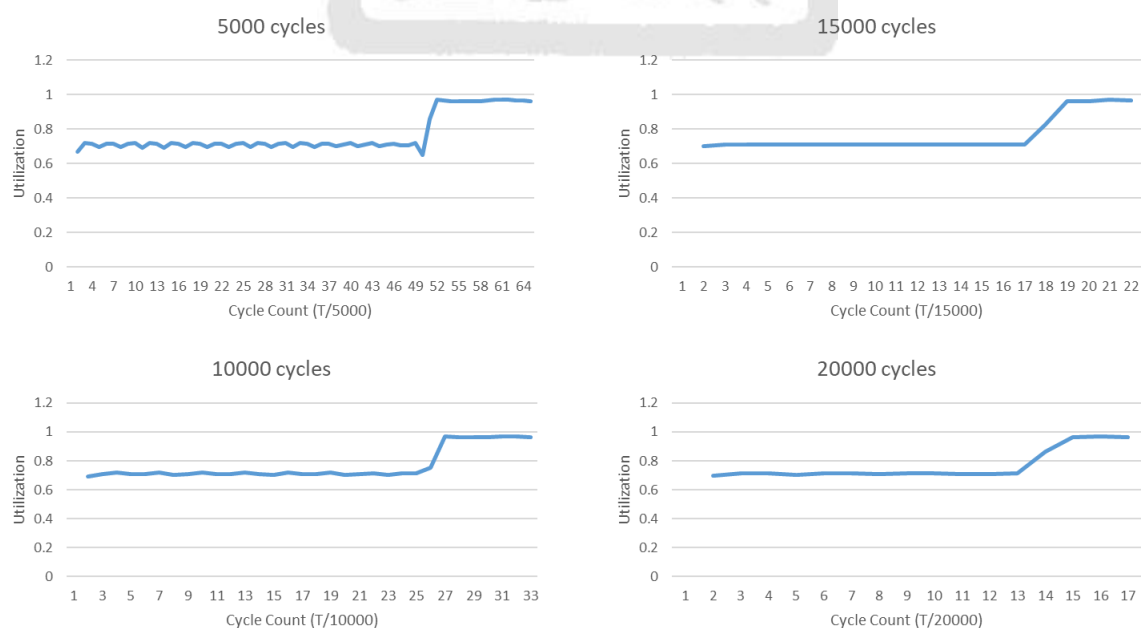


圖 5-4 不同取樣時間間隔對於使用率趨勢之影響

圖 5-5 顯示趨勢判定時間間隔在運行 SYR2K 測試程式時，分別於不同 cycles 中使用率的狀況，在時間間隔選擇的原則與上述相同，最終趨勢判定時間間隔為 40000 cycles。

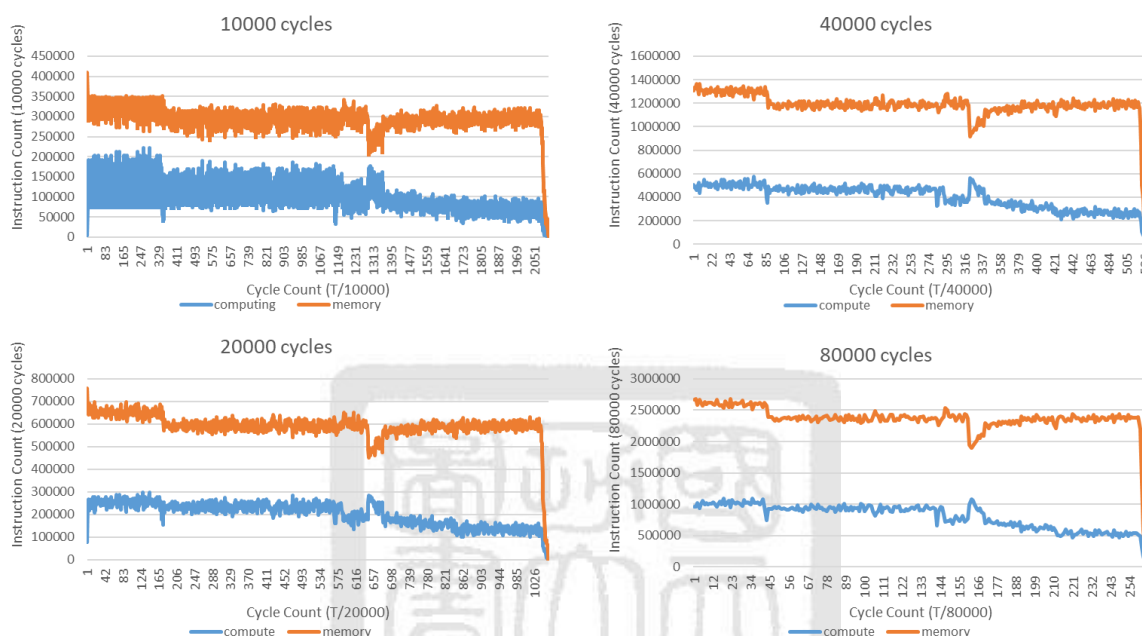


圖 5-5 不同趨勢判定時間間隔對於使用率趨勢之影響

另外一個對於功率管理而言重要的參數是使用率的高低閾值，分析了不同支測試程式的平均使用率狀況將結果列於圖 5-6 中之柱狀圖。在觀察多支測試程式之後將高低閾值分別訂為 0.8 及 0.5，藉由這樣的分界可以明顯的看出這些測試程式分別屬於運算核心使用密集、記憶體存取密集以及趨勢不穩定的狀況，進而在運行過程中可以動態偵測硬體使用率狀況而進行功率管理。圖 5-6 中的虛線區塊代表閾值可調整的範圍，高低閾值個別可調整的範圍是上下共 10% 的範圍。

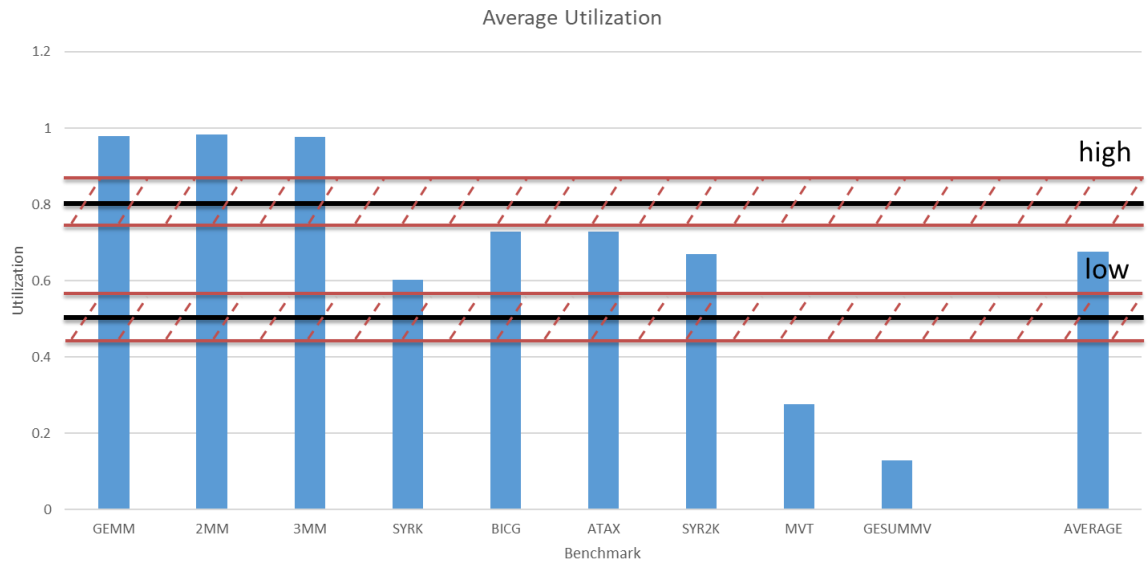


圖 5-6 多支測試程式之平均使用率

圖 5-7 及圖 5-8 分別是本論文所提出之方法與文獻[7]中所提出之 DTA(Data-Traffic Aware)方法之功耗(Power)與效能(Performance)比較圖，比較基準是在所有方法皆有使用時脈閘控機制於其中的情況下。

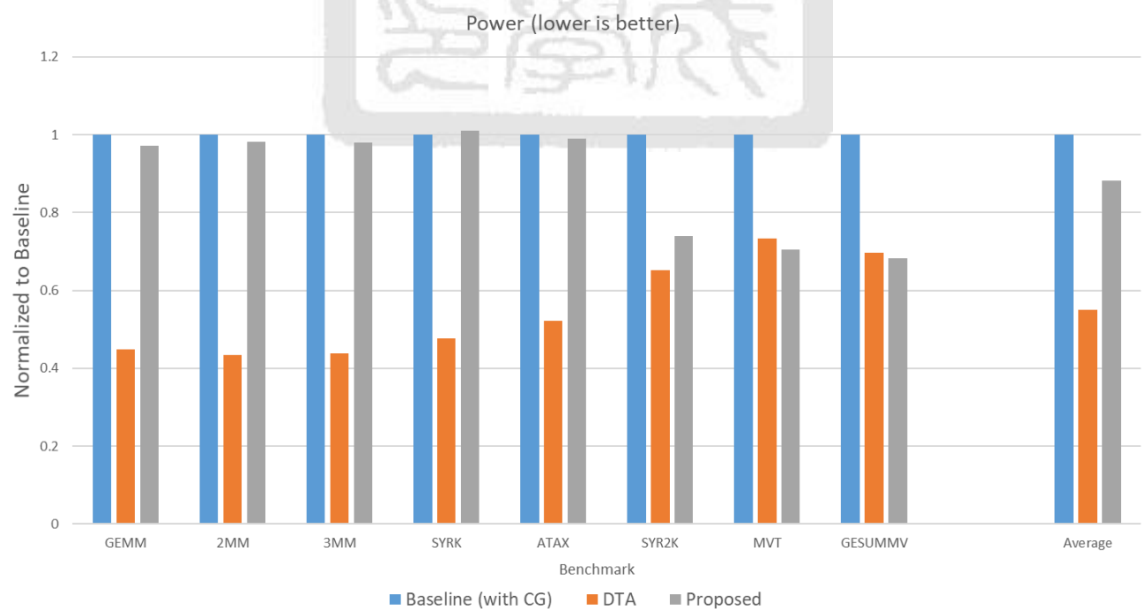


圖 5-7 功耗比較圖

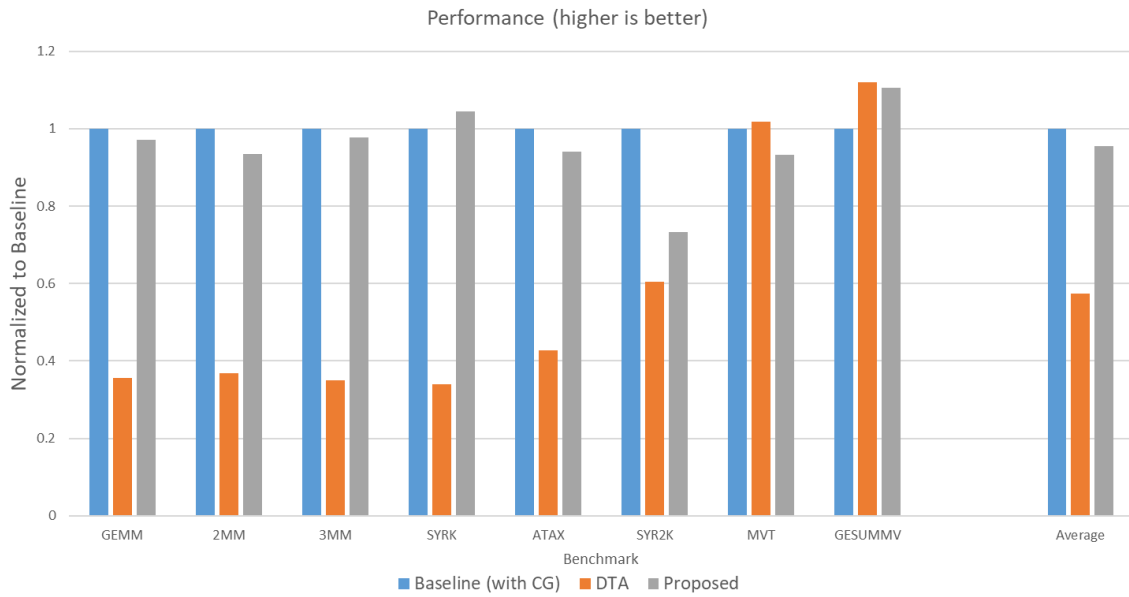


圖 5-8 效能比較圖

整體而言，採用了低功耗技術以及功率管理方法後的功率及效能表現如圖 5-9 及圖 5-10。圖中的比較基準為平台原始的結果，也就是無時脈閘控及功率管理機制的情況，而 DTA 和 Proposed 則是皆有使用時脈閘控機制於其中的情況。圖 5-11 則是整體能量的比較圖。

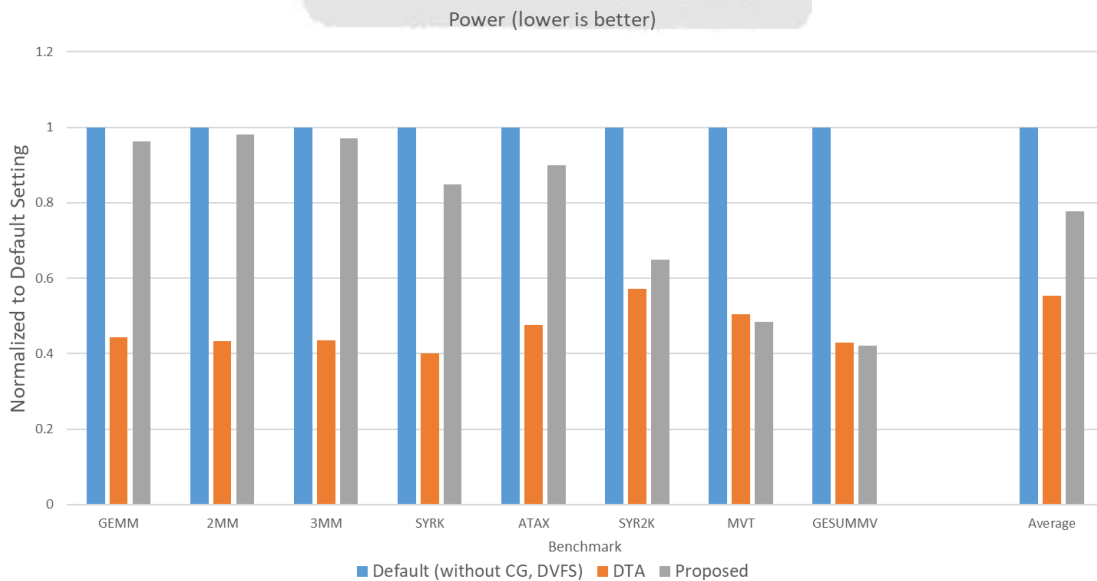


圖 5-9 整體功耗比較圖

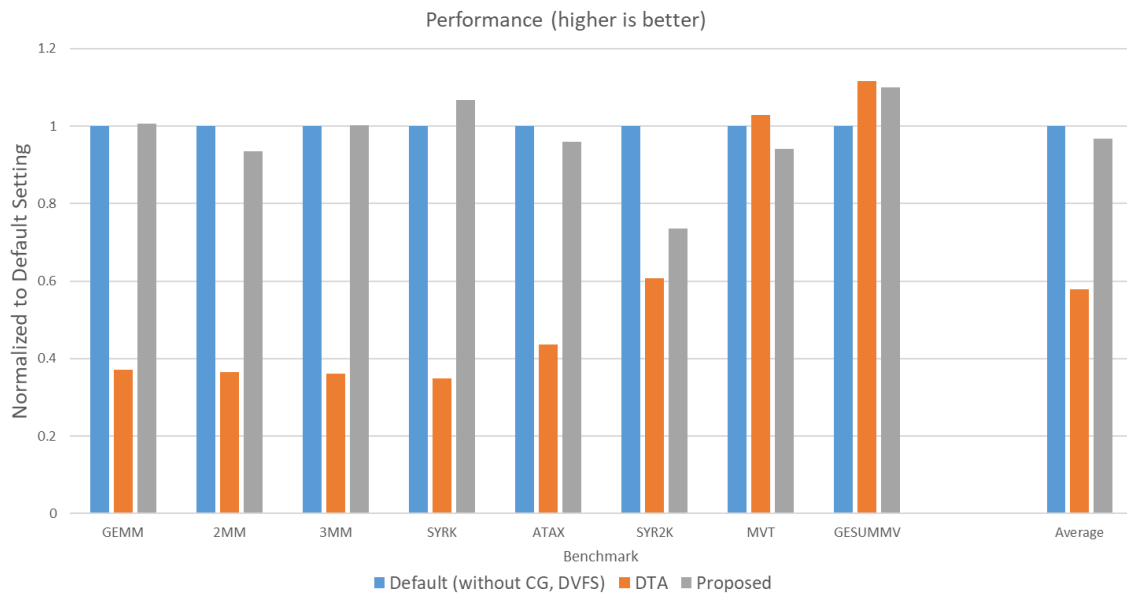


圖 5-10 整體效能比較圖

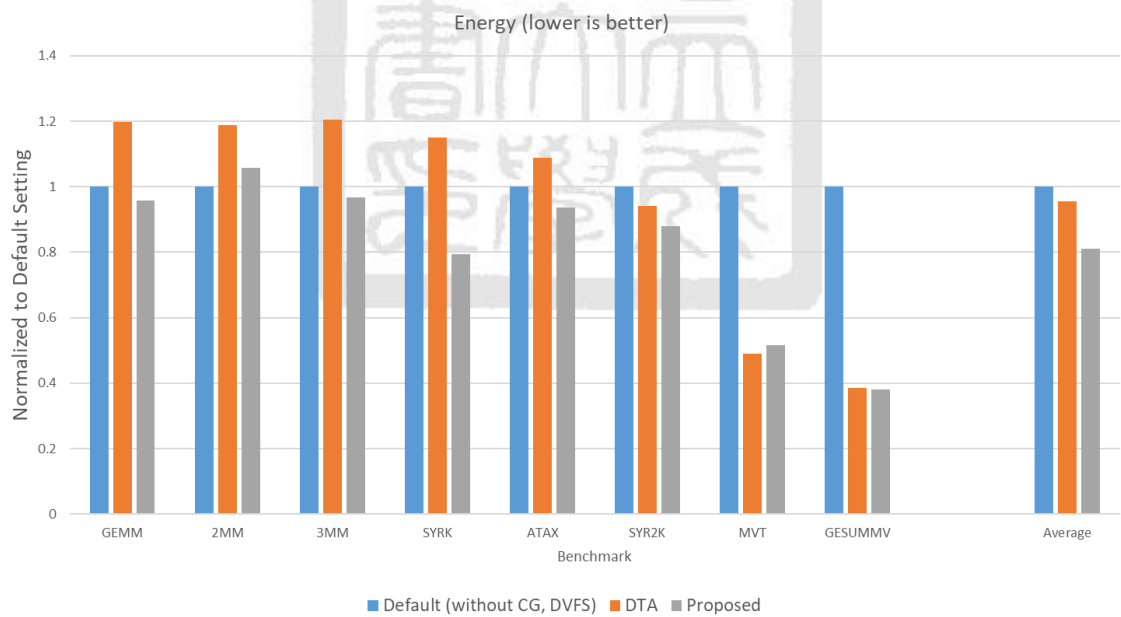


圖 5-11 整體能量比較圖

5.3 額外硬體付出

對於本論文所提出之功率管理而言，需要額外的 PLL 調整頻率、DC-DC converter 調整電壓以及 PMU(Power Management Unit)來管理整體演算法的進行。在分析額外硬體付出的部分，PLL 方面參考了文獻[16]做為 PLL 功率消耗的參考，整體來說在 40nm 製程之下 PLL 本身的功率消耗大約為 2.8mW，對於整體通用計算圖形處理器而言的功率代價(power overhead)約為 0.2%；而 DC-DC converter 方面則是參考 TSMC Austin 於文獻[17]中提出的 DC-DC converter，其於 40nm 製程之下，DC-DC converter 的功率消耗約為 135 μ W，對於整體通用計算圖形處理器而言的功率代價(power overhead)約為 0.01%，而在調整電壓時需要一段時間來穩定電壓，向上調整時為 35 μ s/V(以本論文所需之調整來說大約需要 1575ns)，而向下調整則是 38 μ s/V(以本論文所需之調整大約需要 1710ns)，在運行多個測試程式之後，平均的時序代價(timing overhead)約為 0.046%；而 PMU 的部分則是將演算法的行為以 Verilog 實現，於 40nm 製程下合成之功率代價約為 1.08mW，功率代價約為 0.077%。對於整體通用計算圖形處理器架構而言皆為可忽略之硬體代價付出。

第 6 章 結論和未來工作

6.1 結論

本論文提出一個考慮通用計算圖形處理器之硬體動態使用率的功率管理辦法，使用時脈閘控的低功耗設計並同時取得硬體動態之使用率情形，再透過動態電壓頻率調整對運算核心或是記憶體部分的電壓頻率進行控制來達到提高能量效率之目的。在演算法運行過程中，可以防止電壓頻率的過度調整以及對於使用率閾值也能動態進行調整，對於整體功率管理而言有更好的擴充性可以適應不同的應用程式。

根據實驗結果，整體而言，本論文所提出之方法平均可以降低 **22.3%** 的功率消耗同時只付出 **3.2%** 的效能代價，也就是可以降低約 **20%** 的能量消耗。而做為實驗對照的 DTA 方法，雖然可以降低 44.7% 的功耗，卻也同時付出了 42.1% 的效能代價。



6.2 未來工作

本論文之未來工作可以朝以下幾點做進一步的發展或是加強。

■ 開發不同的功率管理與新的低功耗設計

當前的功率管理演算法考慮硬體使用率為功率管理的判斷依據，未來可以多考慮其他運作特色進行功率管理，加強整體功率管理的趨勢判斷。另外也可以權衡是否能使用其他低功耗設計如電源閘控、Multi- V_{th} Cell 等技術來進一步的提高整體通用計算圖形處理器的能量效率。

■ 考量不同的系統微架構和應用程式

在未來可以根據平台中系統微架構的改動而修正目前的演算法，或是可以使用更複雜的應用程式來檢視當前演算法的效果，再根據軟硬體中遇到的問題來進一步改善演算法。

參考文獻

- [1] VARIANT Market Research, "Edge Computing Market," [Online]. Available: <https://www.variantmarketresearch.com/report-categories/information-communication-technology/edge-computing-market>. [Accessed: 14-Jun-2019].
- [2] Y. Su, J. Jheng, D. Chen and C. Chen, "Development of an Open ISA GPGPU for Edge Device Machine Learning Applications," in *Proc. 2019 Eleventh International Conference on Ubiquitous and Future Networks (ICUFN)*, Zagreb, Croatia, 2019, pp. 214-217.
- [3] Synopsys Inc., "Expanding the Synopsys PrimeTime Solution with Power Analysis", [Online] Available: https://www.eit.lth.se/fileadmin/eit/courses/etin35/PrimeTime/PrimeTime_Slides.pdf [Access: 19-Jun-2019]
- [4] D. Foley *et al.*, "A Low-Power Integrated x86-64 and Graphics Processor for Mobile Computing Devices," *IEEE Journal of Solid-State Circuits*, vol. 47, no. 1, pp. 220-231, Jan. 2012.
- [5] S. R. Gutta, D. Foley, A. Naini, R. Wasmuth and D. Cherepacha, "A low-power integrated x86-64 and graphics processor for mobile computing devices," in *Proc. 2011 IEEE International Solid-State Circuits Conference*, San Francisco, CA, 2011, pp. 270-272.
- [6] P. Meinerzhagen *et al.*, "An energy-efficient graphics processor featuring fine-grain DVFS with integrated voltage regulators, execution-unit turbo, and retentive sleep in 14nm tri-gate CMOS," in *Proc. 2018 IEEE International Solid - State Circuits Conference - (ISSCC)*, San Francisco, CA, 2018, pp. 38-40.

- [7] L. Chiou, C. Yang and C. Chang, "A Data-Traffic Aware Dynamic Power Management for General-Purpose Graphics Processing Units," in *Proc. 2019 IEEE International Symposium on Circuits and Systems (ISCAS)*, Sapporo, Japan, 2019, pp. 1-5.
- [8] C-K. Yang, "A Data-Traffic Aware Dynamic Power Management for General-Purpose Computing on Graphics Processing Units with Dynamic Voltage and Frequency Scaling," M.S. thesis, EE, National Cheng Kung University, Tainan, Taiwan, 2017.
- [9] A. Bakhoda, G. Yuan, Wilson W-L. Fung, H. Wong, Tor M. Aamodt, "Analyzing CUDA Workloads Using a Detailed GPU Simulator," in *Proc. 2009 IEEE International Symposium on Performance Analysis of Systems and Software*, Boston, MA, 2009, pp. 163-174.
- [10] H-Y. Chen, "An HSAIL ISA Conformed GPU Platform," M.S. thesis, CCE, National Cheng Kung University, Tainan, Taiwan, 2015.
- [11] K-C. Hsu, "Performance Prediction Model on HSA-Compatible General-Purpose GPU System," M.S. thesis, CCE, National Cheng Kung University, Tainan, Taiwan, 2016.
- [12] W-S. Hsieh, "Design of Cycle-accurate SIMT Core and Implementation," M.S. thesis, CCE, National Cheng Kung University, Tainan, Taiwan, 2016.
- [13] J-H. Jheng, "Micro-Architecture Optimization of HSA-Compatible GPU," M.S. thesis, CCE, National Cheng Kung University, Tainan, Taiwan, 2018.
- [14] HSA Foundation, "HSA Programmer's Reference Manual: HSAIL Virtual ISA and Programming Model, Compiler Writer, and Object Format (BRIG)," [Online] Available: <http://www.cs.nthu.edu.tw/~ychung/slides/HSA/HSA-PRM-1.02.pdf>. [Accessed: 14-Jun-2019].
- [15] Micron Technology, Inc., "Power Calcs", [Online] Available: https://www.micron.com/-/media/client/global/documents/products/power-calculator/ddr3_ddr3l_power_calc.xlsm?la=en. [Access: 24-Jun-2019]
- [16] Y. Hengzhou, G. Yang and M. Zhuo, "A 40nm/65nm process adaptive low jitter phase-locked loop," in *Proc. 2014 International Symposium on Integrated Circuits (ISIC)*, Singapore, 2014, pp. 500-503.

- [17] J. Shi, Y. Hsu, E. Soenen, A. Roth and J. Gaither, "A wide-range DC/DC converter with 2nd order digital compensation and direct battery connection in 40nm CMOS," in *Proc. 2011 IEEE Custom Integrated Circuits Conference (CICC)*, San Jose, CA, 2011, pp. 1-4.
- [18] J. Lee, V. Sathisha, M. Schulte, K. Compton and N. S. Kim, "Improving Throughput of Power-Constrained GPUs Using Dynamic Voltage/Frequency and Core Scaling," in *Proc. 2011 International Conference on Parallel Architectures and Compilation Techniques*, Galveston, TX, 2011, pp. 111-120.
- [19] J. Guerreiro, A. Ilic, N. Roma and P. Tomas, "GPGPU Power Modeling for Multi-domain Voltage-Frequency Scaling," in *Proc. 2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Vienna, 2018, pp. 789-800.
- [20] K. Kasichayanula, D. Terpstra, P. Luszczek, S. Tomov, S. Moore and G. D. Peterson, "Power Aware Computing on GPUs," in *Proc. 2012 Symposium on Application Accelerators in High Performance Computing*, Chicago IL, 2012, pp. 64-73.
- [21] Z. Lu and Y. Yao, "Thread Voting DVFS for Manycore NoCs," in *IEEE Transactions on Computers*, vol. 67, no. 10, pp. 1506-1524, 1 Oct. 2018.
- [22] A. Majumdar, L. Piga, I. Paul, J. L. Greathouse, W. Huang and D. H. Albonesi, "Dynamic GPGPU Power Management Using Adaptive Model Predictive Control," in *Proc. 2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Austin, TX, 2017, pp. 613-624.
- [23] A. Pathania, Q. Jiao, A. Prakash and T. Mitra, "Integrated CPU-GPU power management for 3D mobile games," in *Proc. 2014 51st ACM/EDAC/IEEE Design Automation Conference (DAC)*, San Francisco, CA, 2014, pp. 1-6.